Entity Framework Core

Meet & Greet

- Participant introductions
- Trainer introduction
- Share experiences with database management
- What do you expect to learn?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Main Course Objectives

- Understand basics of Entity Framework Core
- Learn how to set up and configure EF Core
- Manage database contexts and perform CRUD operations
- Configure relationships and optimize queries
- Master database migrations and schema management
- Utilize advanced features and deploy EF Core applications

Agenda

- Day 1
  1. Understand EF Core
  2. Entity Configuration
  3. Relationships
- Day 2
  1. Querying Data
  2. CRUD Operations
  3. Advanced SQL
- Day 3
  1. Migrations
  2. Logging
  3. Testing
  4. Performance

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Rules of engagement

- 🕛 Training hours
- 🥪 Lunches
- 📱 Phones
- 📖 Training material
- ❓ Evaluation
- 👉 It's your course

Understanding EF Core

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

OO vs Relational

Object Oriented model

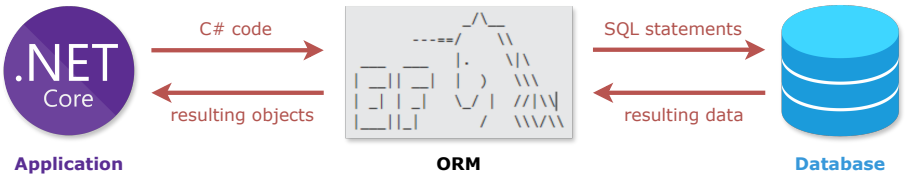
- Object Identity
- Relations via physical pointers
- Objects contain collections
- No concept of permanent storage
- Single user access
- Use OO-language (imperative)

Relational model

- Rows identified by Primary Key
- Relations via Foreign Key
- Flat tables (no repeating groups)
- Only permanent storage
- Multi user access
- Use SQL (declarative)

Object-Relational Mapper ORM

- Bridge between OO model and relational model

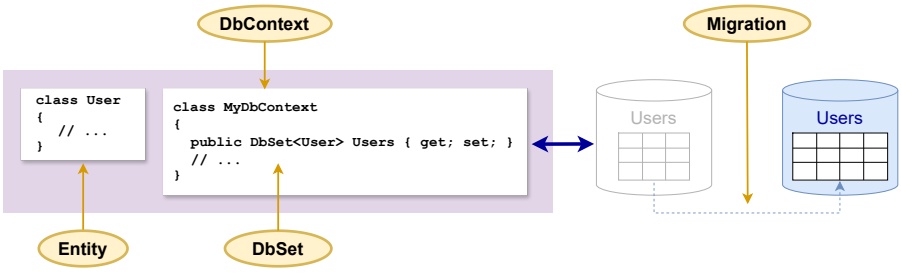


© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Selling points EF Core

- Lightweight, open-source and cross-platform
- Supports LINQ for querying
- Built-in support for migrations
- Support for multiple database providers

Meet the Core Players



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Map the Core Players 🤔

1. Mechanism for updating the database schema
2. Class that maps to a database table
3. Main class interacting with the database
4. Collection of entities of a specific type

Setting Up EF Core

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Project Structure

- Create folders for **Models** and **DAL** (Data Access Layer)
- Add necessary references and dependencies

Entity Class

- Represents a table in the database

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

Customers

Id	Name	Email
1	Ann	a.harvey @ab.co.uk
2	Ben	b.locklear @ab.co.uk
3	Chris	c.hidges@ab.co.uk

? Ever heard of a POCO class?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Install EF Core

- NuGet Package:

  Microsoft.EntityFrameworkCore.SqlServer

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

Providers

Each `DbContext` instance can use **one provider**

DB System	NuGet package
	Npgsql.EntityFrameworkCore.PostgreSQL
Microsoft SQL Server	Microsoft.EntityFrameworkCore.SqlServer
	Microsoft.EntityFrameworkCore.Cosmos
SQLite	Microsoft.EntityFrameworkCore.Sqlite

PostgreSQL, MySQL/MariaDB and Oracle providers available - not provided by Microsoft

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

`DbContext` class

- Central point for database interaction
- Inherits from `DbContext`

```
public class AppDbContext : DbContext
{
  public DbSet<Customer> Customers { get; set; }
  public DbSet<Order> Orders { get; set; }

  public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
}
```

DbSet properties

- Define a `DbSet` property for each entity
- Entities (aka models) are C# classes

```
public class AppDbContext : DbContext
{
  public DbSet<Customer> Customers { get; set; }
  public DbSet<Order> Orders { get; set; }

  public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Constructor

- Takes a `DbContextOptions<AppDbContext>` parameter
- Pass this to the base `DbContext` class

```
public class AppDbContext : DbContext
{
  public DbSet<Customer> Customers { get; set; }
  public DbSet<Order> Orders { get; set; }

  public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
}
```

DbContext initialization

In `Main()`:

- Create `DbContextOptions` object that specifies the db connection details

```
var contextOptions = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer("YourConnectionStringHere")
    .Options;

using (var context = new AppDbContext(contextOptions))
{
  // communicate with db
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Connection strings

- PostgreSQL:

```
"Host=localhost;Port=5432;Database=Test;Username=postgres;Password=password"
```

- Local db:

```
@"Server=(localdb)\mssqllocaldb;Database=Test;ConnectRetryCount=0"
```

- Local installation SQL Server Developer:

```
"Server=localhost;Database=Test;Trusted_Connection=True;TrustServerCertificate=True"
```

- Local installation SQL Server Express:

```
@"Server=.\SQLExpress;Database=Test;Trusted_Connection=True;TrustServerCertificate=True"
```

- Docker image with SQL Server Developer:

```
"Server=127.0.0.1,1433;Database=Test;User=sa;Password=123;TrustServerCertificate=True"
```

DbContext initialization

- Create instance of the context passing the connection details

```
var contextOptions = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer("YourConnectionStringHere")
    .Options;

using (var context = new AppDbContext(contextOptions))
{
  // communicate with db
}
```

? What is the purpose of `using` here?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Build Database and Schema

- For testing or rapid prototyping use

```
using (var context = new AppDbContext(contextOptions))
{
  // communicate with db
  context.Database.EnsureDeleted();
  context.Database.EnsureCreated();
}
```

  - `.EnsureCreated()` - only create db and schemas if db doesn't exist yet or is empty

- On production use **migrations** as we'll see later

Add data to table

Can be done via:

```
var customer = new Customer { Name = "Steve Harvey", City = "Veenendaal" };
context.Customers.Add(customer);
context.SaveChanges();
```

More on adding data later ...

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Retrieve rows from table

- Use the `DbSet` property

```
var customers = context.Customers;   // 👈

foreach (var customer in customers)
{
    Console.WriteLine($"{customer.Id} {customer.Name}");
}
```

Connection string in config file

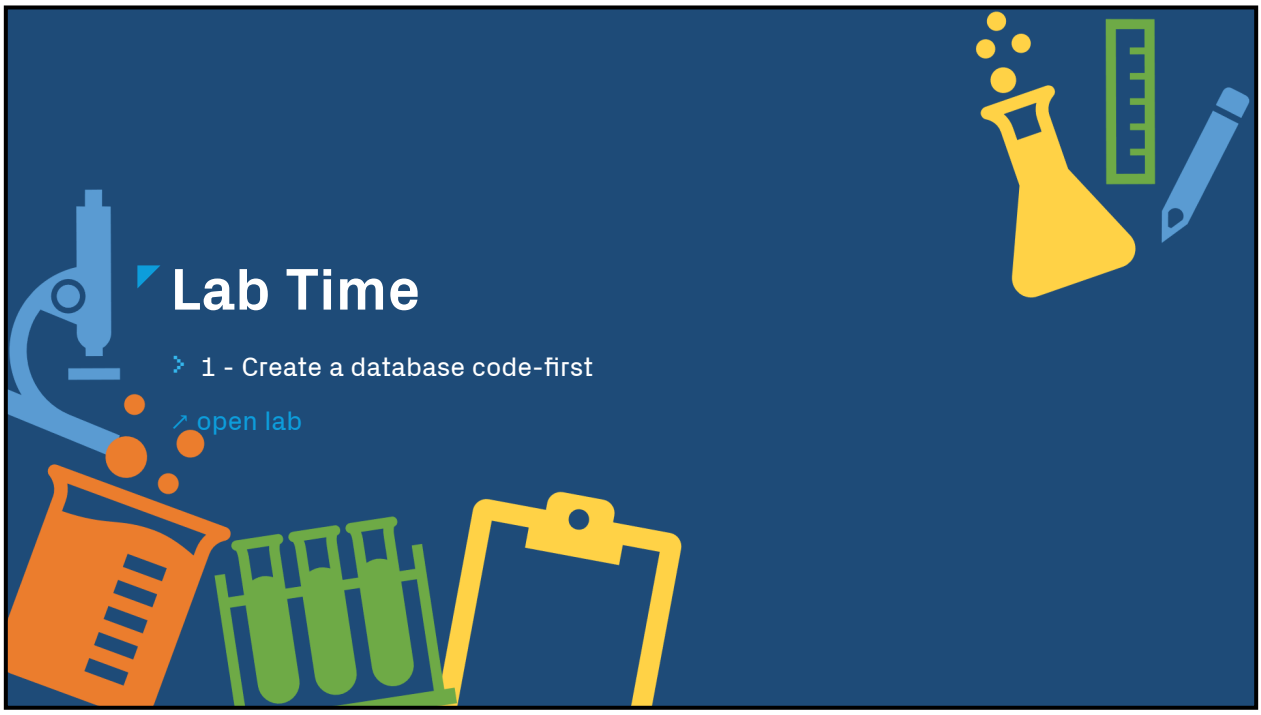
1. Install NuGet package: `Microsoft.Extensions.Configuration.Json`
2. Add to *`appsettings.json`*

```
{
  "ConnectionStrings": {
    "DefaultConn": "YourConnectionStringHere"
  }
}
```

3. Read connection string:

```
var builder = new ConfigurationBuilder().AddJsonFile("appsettings.json");
var configuration = builder.Build();

var contextOptions = new DbContextOptionsBuilder<WebShopDbContext>()
      .UseSqlServer(configuration.GetConnectionString("DefaultConn"))
      .Options;
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



Key Takeaways

- **EF Core**
  - Lightweight, open-source, cross-platform ORM
  - Supports LINQ for querying & automated change tracking
  - Available via EF Core NuGet packages
- **DbContext Class**
  - Central point for database interaction
  - Inherits from `DbContext`
  - Contains `DbSet` properties for each entity
  - Constructor accepts `DbContextOptions` containing connection string

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Entity Configuration

Configuration Strategies

- Conventions
- Data annotations
- Fluent API

- Built-in Conventions
- Data annotations
- Fluent API

What does the EF Core principle **Convention over Configuration** mean?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Built-in conventions

- Enabled by default:
  - Public properties become table columns
  - Property `int Id` becomes autoincremented key
  - Many more ...
- (classes implementing the `IConvention` interface define these model-building behaviors)

Data Annotations

- Simple and declarative

Data annotation

```
public class Customer
{
    public int Id { get; set; }

    [MaxLength(500)]
    public string Name { get; set; }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Data Annotations

- Reference
  - `System.ComponentModel.DataAnnotations.Schema`
  - `System.ComponentModel.DataAnnotations`
- Many data annotations exist - most important explained in this course
  - `[Table]`
  - `[Key]`
  - `[Required]`
  - `[MaxLength]`
  - `[Index]`
  - `[NotMapped]`
  - …

? Can you guess the use of each of the above data annotations?

Fluent API

More flexible and powerful

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .Property(c => c.Name)
        .HasMaxLength(500);
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Entity Types

Types included in the model

1. in `DbSet` properties

```
public DbSet<Author> Authors { get; set; }        // Author = model, Authors = table
```

2. in navigation properties of other entities (later more)

```
public class Author
{
    public string Name { get; set; }

    public ICollection<Blog> Blogs { get; set; }  // Blog = model, Blogs = table
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Table and Schema Name

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .ToTable("Blogs", "blogging");
}
```

or

Data annotation

```
[Table("Blogs", Schema="blogging")]
public class Blog
{
    public int BlogId { get; set; }
    public string Url { get; set; }
}
```

Default Schema

- a *default* schema can be specified on the `modelBuilder`

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.HasDefaultSchema("blogging);
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Included Properties

Convention

- Include all public properties with a getter and a setter in the model:

```
public class Blog
{
    public int BlogId { get; set; }
    public string Url { get; set; }
}
```

Exclude Properties

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .Ignore(b => b.LoadedFromDatabase);
}
```

or

Data annotation

```
public class Blog
{
    [NotMapped]
    public DateTime LoadedFromDatabase { get; set; }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Entity Properties

Column Name

- Name
- Type
- Length
- Required/Optional
- Keys
- Generated

Convention

- Entity properties are mapped to table columns having the **same name**

```
public class Blog
{
    public int Id { get; set; }
    public string Url { get; set; }
}
```

Blogs

Id	Url
1	https://tinyurl.com/abc123
2	https://tinyurl.com/def456
3	https://tinyurl.com/ghi789

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Column Name

- Name
- Type
- Length
- Required/Optional
- Keys
- Generated

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
   modelBuilder.Entity<Blog>()
       .Property(b => b.BlogId)
       .HasColumnName("blog_id");
}
```

or

Data annotation

```
public class Blog
{
    [Column("blog_id")]
    public int BlogId { get; set; }
}
```

Column Data Type

- Name
- Type
- Length
- Required/Optional
- Keys
- Generated

Convention

- Database provider determines the data type based on
  - **.NET type** of the property
  - Other **metadata** (e.g. max length, primary key)
- SQL Server examples:
  - `DateTime` → `datetime2(7)`
  - `String` → `nvarchar(max)` or `nvarchar(450)` (for keys)

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Column Data Type

- Name
- **Type**
- Length
- Required/Optional
- Keys
- Generated

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
      .Property(b => b.Url)
      .HasColumnType("varchar(200)");

    modelBuilder.Entity<Blog>()
      .Property(b => b.Rating)
      .HasColumnType("decimal(5,2)");
}
```

or

Data annotation

```
public class Blog {
    [Column(TypeName = "varchar(200)")]
    public string Url { get; set; }

    [Column(TypeName = "decimal(5,2)")]
    public decimal Rating { get; set; }
}
```

Maximum Length

- Name
- Type
- **Length**
- Required/Optional
- Keys
- Generated

- Guides the database provider on column data type selection
- Applies only to arrays like `string` and `byte[]`

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .Property(b => b.Url)
        .HasMaxLength(500);
}
```

or

Data annotation

```
public class Blog {
    [MaxLength(500)]
    public string Url { get; set; }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Precision and Scale

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .Property(b => b.Score)    // assume decimal
        .HasPrecision(14, 2);      // max. 14 digits, max. 2 decimal places

    modelBuilder.Entity<Blog>()
        .Property(b => b.LastUpdated)
        .HasPrecision(3);          // max. 3 digits to express fractions of seconds
}
```

or

Data annotation

```
public class Blog
{
    [Precision(14, 2)]
    public decimal Score { get; set; }
    [Precision(3)]
    public DateTime LastUpdated { get; set; }
}
```

Required & Optional Properties

- **required** properties → **non-nullable** columns
  `int`, `decimal`, `bool`, ...
- **optional** properties → **nullable** columns
  `int?`, `decimal?`, `bool?`, ...

- if disabled then *all* .NET reference types are optional
  `string` → **optional** property
- if enabled then .NET reference types follow C# nullability
  `string` → **required** property
  `string?` → **optional** property

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Explicit Configuration

- Name
- Type
- Length
- Required/Optional
- Keys
- Generated

- A property - optional by convention - can be configured as required:

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .Property(b => b.Url)
        .IsRequired();
}
```

or

Data annotation

```
public class Blog
{
    [Required]
    public string Url { get; set; }
}
```

Primary Key

- Name
- Type
- Length
- Required/Optional
- Keys
- Generated

Convention

- A property named `Id` or `<TypeName>Id` will be the primary key

```
internal class Car
{
    // 🔑 Primary Key
    public string Id { get; set; }

    public string Make { get; set; }
    public string Model { get; set; }
}
```

```
internal class Truck
{
    // 🔑 Primary Key
    public string TruckId { get; set; }

    public string Make { get; set; }
    public string Model { get; set; }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Primary Key

- Configure a **single property** as the primary key of an entity:

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Car>()
        .HasKey(c => c.LicensePlate);
}
```

or

Data annotation

```
internal class Car
{
    [Key]
    public string LicensePlate { get; set; }
}
```

Composite Key

Configure **multiple properties** as a composite key:

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Car>()
        .HasKey(c => new { c.State, c.LicensePlate });
}
```

or

Data annotation

```
[PrimaryKey(nameof(State), nameof(LicensePlate))]
internal class Car
{
    public string State { get; set; }
    public string LicensePlate { get; set; }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Auto increment

- Name
- Type
- Length
- Required/Optional
- Keys
- **Generated**

Convention

- For non-composite primary keys (`short`, `int`, `long` or `Guid`) DB provider auto-generates values if none are provided

Data annotation

```
[Key]
public int LicensePlate { get; set; }
```

```
LicensePlate int IDENTITY(1, 1) PRIMARY KEY
```

Default Constraint

- Name
- Type
- Length
- Required/Optional
- Keys
- **Generated**

- Builds a default constraint in the database

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
  modelBuilder.Entity<Product>()
    .Property(p => p.Stock)
    .HasDefaultValue(100);            // 👈

  modelBuilder.Entity<Product>()
    .Property(p => p.CreatedAt)
    .HasDefaultValueSql("CURRENT_DATE")  // 👈
}
```

❓ What is the purpose of a **default constraint** ?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

🔧 Computed Column

- Builds a computed column in the database
- Optionally, store the result to improve read performance

- Name
- Type
- Length
- Required/Optional
- Keys
- **Generated**

Fluent API

```
modelBuilder.Entity<Order>()
  .Property(p => p.TotalPrice)
  .HasComputedColumnSql("[Quantity] * [Price]", stored: true);  // 👈
```

❓ Will EF Core include **TotalPrice** in `INSERT` and `UPDATE` statements?

DB-generated values

- Only control **when** values are generated, three options:

- Name
- Type
- Length
- Required/Optional
- Keys
- **Generated**

Fluent API

```
modelBuilder.Entity<Invoice>()
  .Property(i => i.InvoiceId)
  .ValueGeneratedNever();           // 1 Application supplies value (no auto-increment)
```

```
modelBuilder.Entity<Patient>()
  .Property(p => p.Created)
  .ValueGeneratedOnAdd();           // 2 Column excluded in INSERTs
```

```
modelBuilder.Entity<Room>()
  .Property(r => r.LastModified)
  .ValueGeneratedOnAddOrUpdate();   // 3 Column excluded in INSERTs and UPDATEs
```

- 2 3: EF Core reads the db-generated value after saving

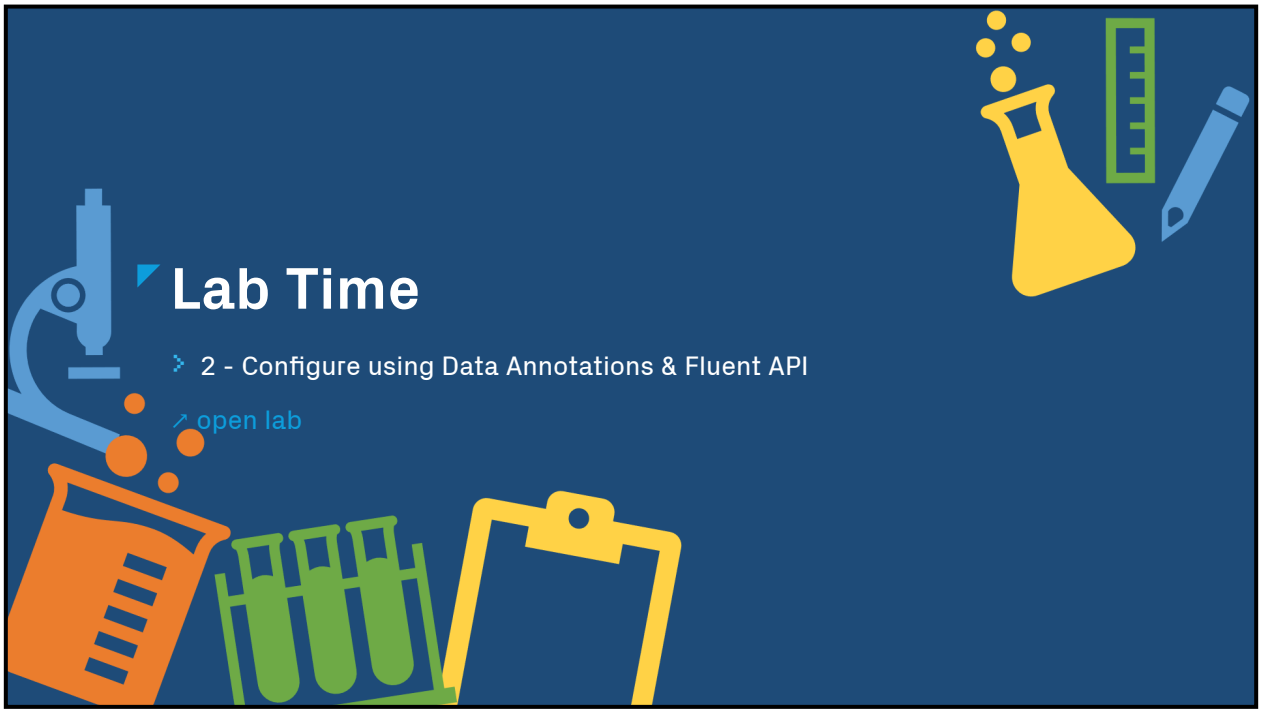
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Fluent API configuration

- `OnModelCreating` can grow excessively!
- Solution - one configuration class per entity:

```
public class BlogConfiguration : IEntityTypeConfiguration<Blog>
{
    public void Configure(EntityTypeBuilder<Blog> builder)
    {
        builder.Property(b => b.Id).ValueGeneratedNever();
        builder.Property(b => b.Url).HasMaxLength(500);
    }
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.ApplyConfiguration(new BlogConfiguration());   // 👆
    // ...
}
```

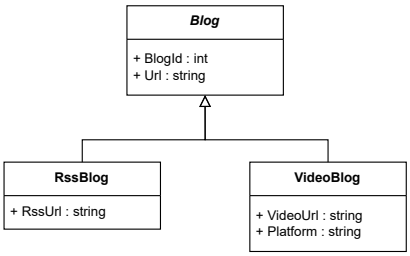


© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Inheritance

Inheritance

- OO programming supports inheritance:




```
public abstract class Blog
{
    public int BlogId { get; set; }
    public string Url { get; set; }
}

public class RssBlog : Blog
{
    public string RssUrl { get; set; }
}

public class VideoBlog : Blog
{
    public string VideoUrl { get; set; }
    public string Platform { get; set; }
}
```

- Relational databases lack inheritance
- EF Core provides 3 strategies to map class hierarchies to db tables

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Strategies

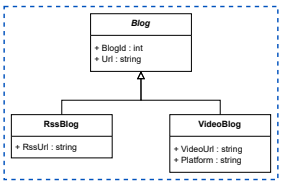


Table per Hierarchy
TPH

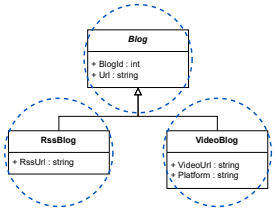


Table per Type
TPT

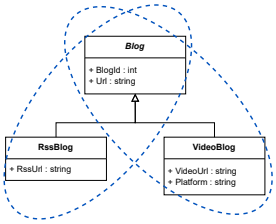
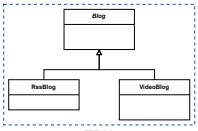


Table per Concrete Type
TPC

Table per Hierarchy

- Default strategy
- **Single table** stores data for all types in the hierarchy
- **Discriminator column** identifies the type of each row

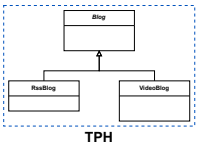


TPH

BlogId	Url	Discriminator	RssUrl	VideoUrl	Platform
1	rss-a1.com	RssBlog	rss-a2.com	NULL	NULL
2	rss-b1.com	RssBlog	rss-b2.com	NULL	NULL
3	video-a1.com	VideoBlog	NULL	video-a2.com	YouTube

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Explicit Discriminator

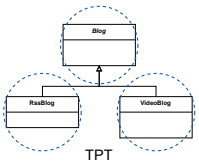


TPH

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .HasDiscriminator<string>("blog_type")
        .HasValue<Blog>("blog_base")
        .HasValue<RssBlog>("blog_rss")
        .HasValue<VideoBlog>("blog_video");
}
```

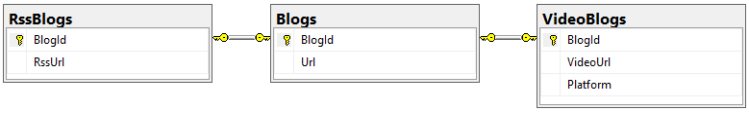
BlogId	Url	blog_type	RssUrl	VideoUrl	Platform
1	rss-a1.com	blog_rss	rss-a2.com	NULL	NULL
2	rss-b1.com	blog_rss	rss-b2.com	NULL	NULL
3	video-a1.com	blog_video	NULL	video-a2.com	YouTube

Table per Type



TPT

- Each type has its own table
- Type-specific properties are stored in their respective tables
- Derived type tables reference the base table via a foreign key



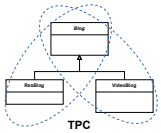
- map every entity manually, or:

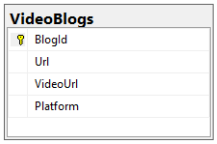
```
modelBuilder.Entity<Blog>().UseTptMappingStrategy();
```

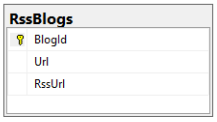
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Table per Concrete Type

- Every **concrete type** is mapped to its own table
- Abstract types do not have tables
- TPC database schemas are denormalized
- Db Sequence is built for auto-increment





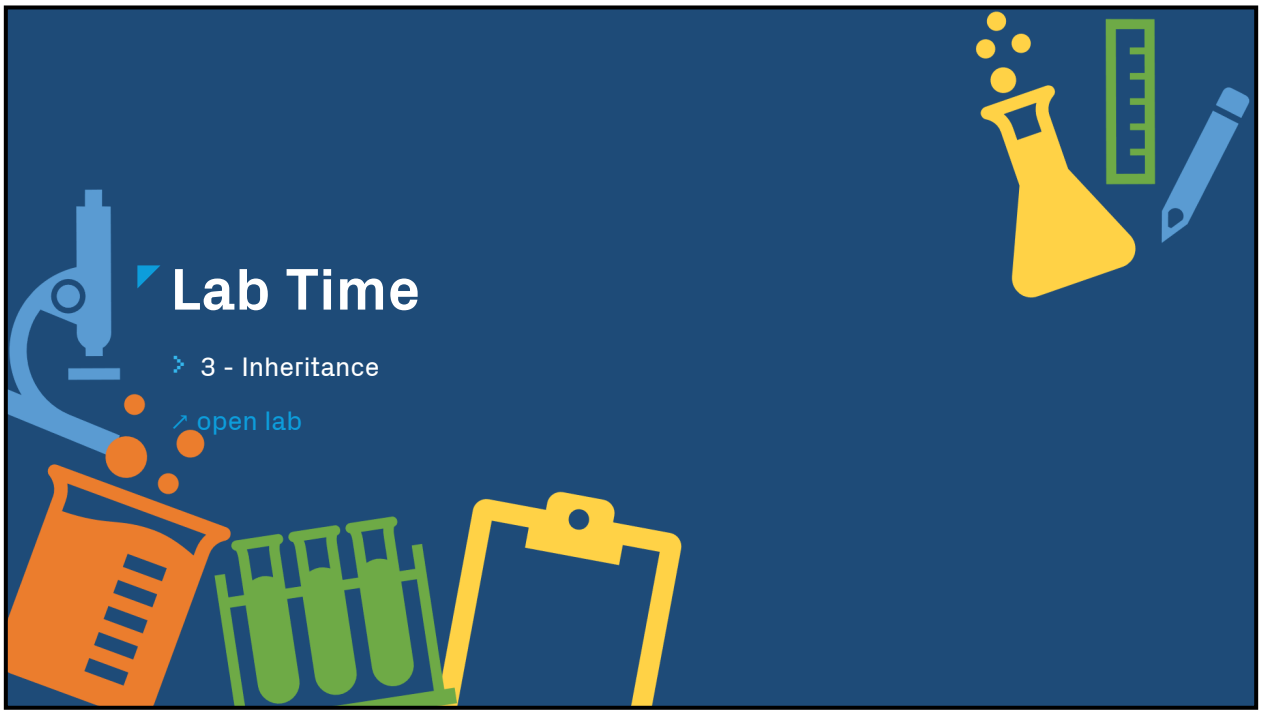



```
modelBuilder.Entity<Blog>().UseTpcMappingStrategy();
```

Strategy Considerations

- Table per Hierarchy
  - ✅ simple and good query performance
  - ❌ many null values and difficult to change schema
- Table per Type
  - ✅ clear normalized schema
  - ❌ complex and slower queries
- Table per Concrete Type
  - ✅ simple and good query performance
  - ❌ retrieve data from all tables in hierarchy more work

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



Key Takeaways

- Entities and entity properties are configured by:
  - Convention rules - default behavior
  - Data Annotations - overrides conventions, declarative
  - Fluent API - overrides all, most flexible
- Support for primary keys and composite keys
- Three value generation strategies:
  - `ValueGeneratedNever()` - application must provide value
  - `ValueGeneratedOnAdd()` - db generates value (exclude column in `INSERT`)
  - `ValuedGeneratedOnAddOrUpdate()` - db generates value (exclude in `INSERT` and `UPDATE`)
- Three inheritance strategies:
  - Table per Hierarchy (TPH) - single table for all classes
  - Table per Type (TPT) - separate table for each class
  - Table per Concrete Type (TPC) - separate table for each concrete class

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

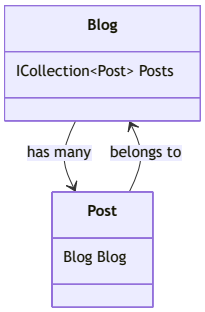
Relationships

Relationships in object models

A **relationship** defines how *two entities relate to each other*

```
public class Blog {
    ...
    public ICollection<Post> Posts { get; }
}
```

```
public class Post {
    ...
    public Blog Blog { get; set; }
}
```



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Database Relationships

Relational databases represent relationships using **foreign keys**

```
CREATE TABLE Blogs (
    Id int NOT NULL IDENTITY,
    CONSTRAINT PK_Blogs PRIMARY KEY (Id)
);

CREATE TABLE Posts (
    Id int NOT NULL IDENTITY,
    CONSTRAINT PK_Posts PRIMARY KEY (Id),
    BlogId int NOT NULL,

    CONSTRAINT FK_Posts_Blogs_BlogId FOREIGN KEY (BlogId)
    REFERENCES Blogs (Id) ON DELETE CASCADE
);
```

? Which field is the foreign key in this code excerpt?

One-to-many

```
public class Blog
{
    public int Id { get; set; }

    public ICollection<Post> Posts { get; }
}
```

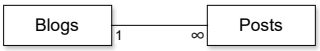
```
public class Post
{
    public int Id { get; set; }

    public int BlogId { get; set; }
    public Blog Blog { get; set; }
}
```

? Which properties will EF Core configure as **primary key** and which as **foreign key**?

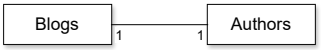
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Relationship Types

- One-to-many
  - Single entity associated with multiple entities



- One-to-one
  - Single entity associated with one other entity



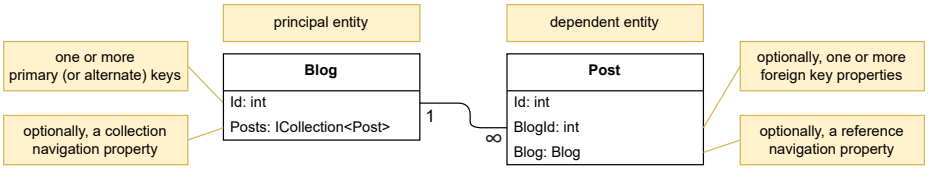
- Many-to-many
  - Single entity associated with one other entity

One-to-many

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Discovery by Convention

- One-to-many
- One-to-one
- Many-to-many



Required vs optional relation

- One-to-many
- One-to-one
- Many-to-many

A relation can be required or optional

- **required** - a `Post` must refer to a `Blog`
- **optional** - a `Post` may refer to a `Blog`

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Required relation

- One-to-many
- One-to-one
- Many-to-many

- by convention

```
public class Blog {
    public int Id { get; set; }
    public ICollection<Post> Posts { get; } = new List<Post>();
}
```

```
public class Post {
    public int Id { get; set; }

    public int BlogId { get; set; }            // 👆 not null
    public Blog Blog { get; set; } = null!;    // 👆 not null
}
```

🔧 Required relation

- One-to-many
- One-to-one
- Many-to-many

- manually

Fluent API

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .HasMany(b => b.Posts)
        .WithOne(p => p.Blog)
        .HasForeignKey(p => p.BlogId)
        .IsRequired();    // 👆 required
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

When to use Fluent API?

- **Discovery by convention fails**
  For example when properties are inconsistent:

```
public class Post {
  public int Id { get; set; }

  public int? BlogId { get; set; }           // 👈 nullable foreign key
  public Blog Blog { get; set; } = null!;    // 👈 non-nullable navigation
}
```

- **Ensure clarity**
  For example when the option *nullable reference types* is disabled

Optional relation

by convention:

```
public class Post {
    public int Id { get; set; }

    public int? BlogId { get; set; }  // 👈
    public Blog? Blog { get; set; }   // 👈
}
```

manually:

Fluent API

```
modelBuilder.Entity<Blog>()
    .HasMany(e => e.Posts)
    .WithOne(e => e.Blog)
    .HasForeignKey(e => e.BlogId)
    .IsRequired(false);   // 👈
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Shadow foreign key

- One-to-many
- One-to-one
- Many-to-many

- = no foreign key defined in model

```
public class Post {
    public int Id { get; set; }

    // -- no foreign key here --
    public Blog Blog { get; set; } = null!;
}
```

- In a one-to-many even created when not configured via Fluent API !
- ✅ Code more object-oriented
- ❌ Harder to understand/debug
- ❌ Cannot be directly accessed in code
- ❌ Issues when serialization needed

Unidirectional relationship

- One-to-many
- One-to-one
- Many-to-many

- Skip navigation property in just one entity:

```
public class Blog {
    public int Id { get; set; }
    // -- no navigation property --
}

public class Post {
    public int Id { get; set; }

    public int BlogId { get; set; }
    public Blog Blog { get; set; } = null!;
}
```

or

```
public class Blog {
    public int Id { get; set; }
    public ICollection<Post> Posts { get; } = [];
}

public class Post {
    public int Id { get; set; }

    public int BlogId { get; set; }
    // -- no navigation property --
}
```

- Discovered by convention to a one-to-many
- Avoids unnecessary navigation when not needed
- Can improve query performance (and serialization)

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Unidirectional relationship

Set manually if discovery by convention fails, or you want to be explicit.

- Assuming `Post` has no navigation property to `Blog`:
- solution 1:

Fluent API

```
modelBuilder.Entity<Blog>()
  .HasMany(e => e.Posts)
  .WithOne()          // 👆 no arguments
  .HasForeignKey(e => e.BlogId)
  .IsRequired();
```

- solution 2:
  - Use generic version: `HasOne<T>()`

Fluent API

```
modelBuilder.Entity<Post>()
  .HasOne<Blog>()    // 👆 generic
  .WithMany(e => e.Posts)
  .HasForeignKey(e => e.BlogId)
  .IsRequired();
```

No navigations at all

- Skip navigation in both entities:

```
public class Blog {
    public int Id { get; set; }
    // -- no navigation property --
}

public class Post {
    public int Id { get; set; }

    public int BlogId { get; set; }
    // -- no navigation property --
}
```

- **Not** discovered by convention

- Two solutions:

Fluent API

```
modelBuilder.Entity<Post>()
  .HasOne<Blog>()          // 👆
  .WithMany()              // 👆
  .HasForeignKey(e => e.BlogId)
  .IsRequired();
```

or

Fluent API

```
modelBuilder.Entity<Blog>()
  .HasMany<Post>()         // 👆
  .WithOne()               // 👆
  .HasForeignKey(e => e.BlogId)
  .IsRequired();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

No Cascade Delete

- One-to-many
- One-to-one
- Many-to-many

- Required relations cascade delete:
  Deleting the principal also deletes its dependents
- To prevent this, configure EF Core to throw an exception:

Fluent API

```
modelBuilder.Entity<Blog>()
  .HasMany(e => e.Posts)
  .WithOne(e => e.Blog)
  .OnDelete(DeleteBehavior.Restrict);   // 👈
```

Self-referencing relation

- One-to-many
- One-to-one
- Many-to-many

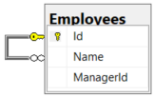
by convention

```
public class Employee
{
    public int Id { get; set; }

    // Optional foreign key property
    public int? ManagerId { get; set; }

    // Optional reference navigation to principal
    public Employee? Manager { get; set; }

    // Collection navigation containing dependents
    public ICollection<Employee> Reports { get; } =
        new List<Employee>();
}
```

manually

Fluent API

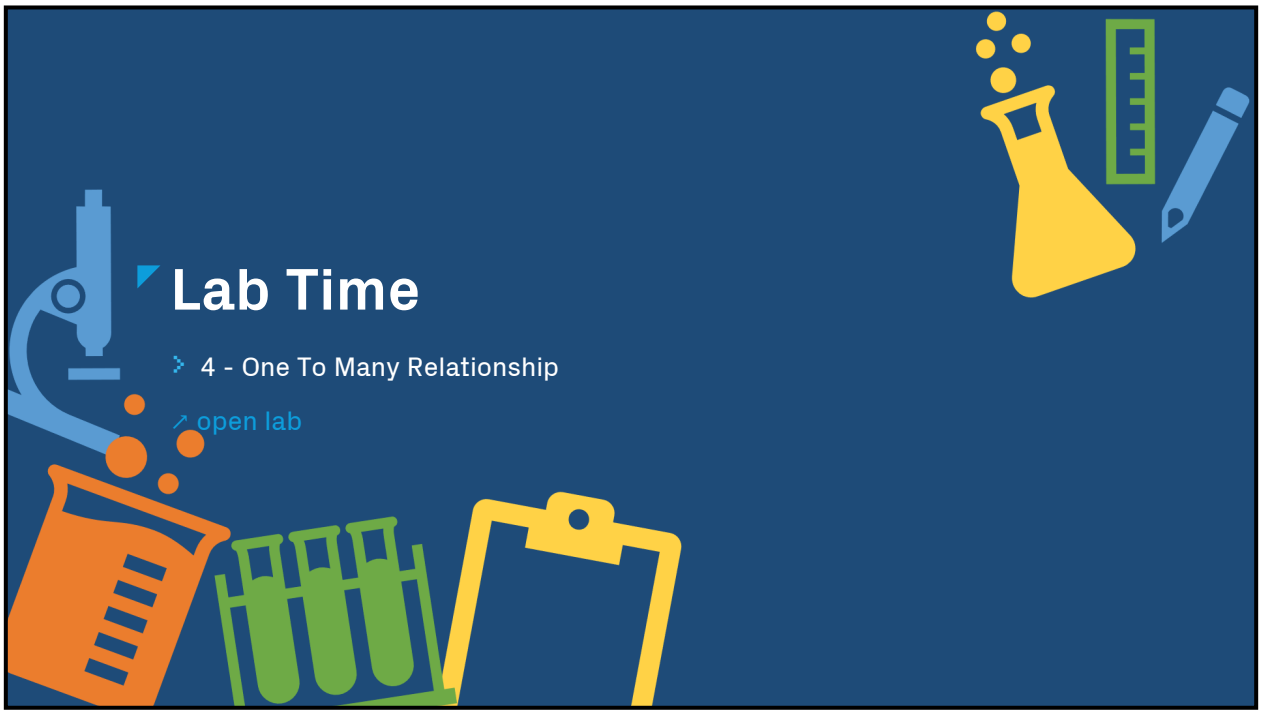
```
modelBuilder.Entity<Employee>()
    .HasOne(e => e.Manager)
    .WithMany(e => e.Reports)
    .HasForeignKey(e => e.ManagerId)
    .IsRequired(false);
```

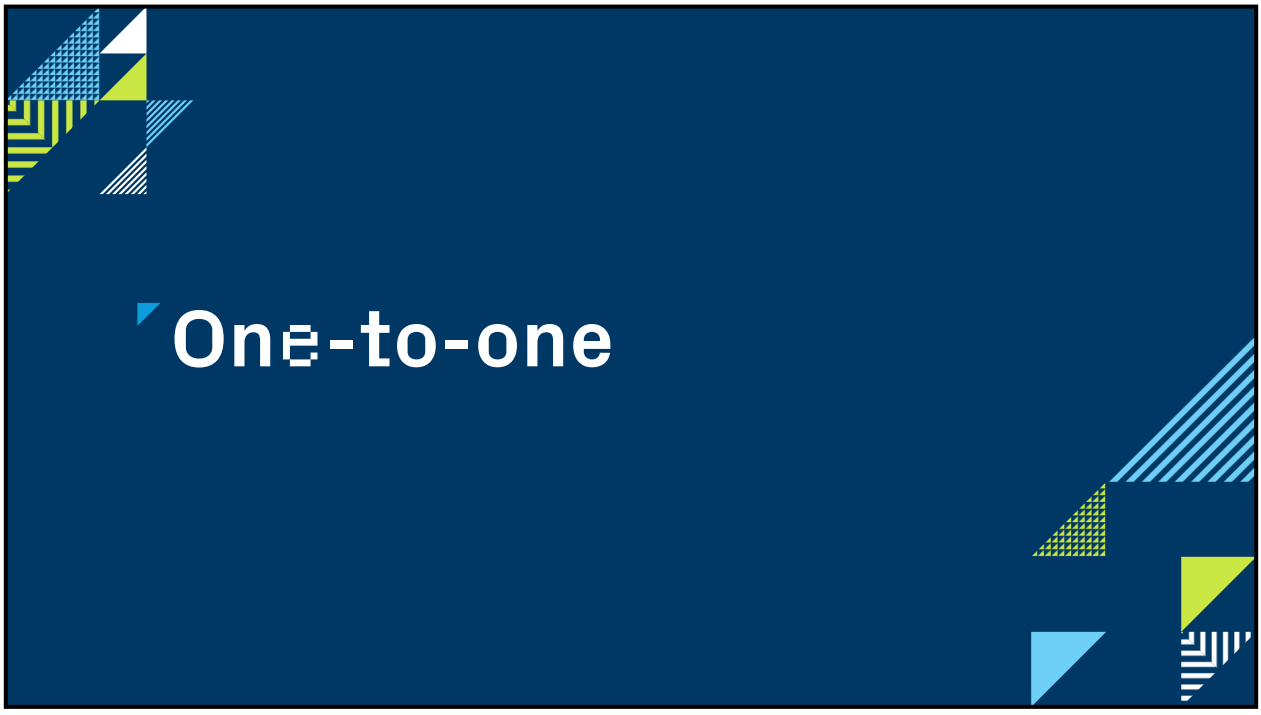


Employees

Id	Name	ManagerId
1	Ann	NULL
2	Ben	1
3	Chris	2
4	Karin	2

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support





© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

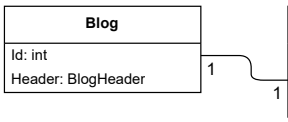
Discovery by Convention

- One-to-many
- One-to-one
- Many-to-many

optionally, one or more foreign key properties

optionally, a reference navigation property

dependent entity



one or more primary (or alternate) keys

principal entity

optionally, a reference navigation property

Try to map the descriptions

Required vs Optional

- One-to-many
- One-to-one
- Many-to-many

Similar logic as one-to-many

by convention:

```
public class BlogHeader {
    public int Id { get; set; }

    public int BlogId { get; set; }
    public Blog Blog { get; set; } = null!;
}
```

manually:

Fluent API

```
modelBuilder.Entity<Blog>()
  .HasOne(e => e.Header)
  .WithOne(e => e.Blog)
  .HasForeignKey<BlogHeader>(e => e.BlogId)
  .IsRequired();
```

How to make the relation from `BlogHeader` to `Blog` *optional*?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

PK-to-PK relationship

Ids of principal and dependent table match

- **Must** be configured manually:

Fluent API

```
modelBuilder.Entity<Blog>()
  .HasOne(e => e.Header)
  .WithOne(e => e.Blog)
  .HasForeignKey<BlogHeader>(e => e.Id);
```

? This relation cannot be nullable, why?

Shadow foreign key

Different compared to one-to-many!

- **Must** be configured manually:

Fluent API

```
modelBuilder.Entity<Blog>()
    .HasOne(e => e.Header)
    .WithOne(e => e.Blog)
    .HasForeignKey<BlogHeader>("BlogId");
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Other

- Unidirectional or no navigations: configure the relationship manually
- Throw an exception, instead of cascade delete:

Fluent API

```
modelBuilder.Entity<Blog>()
    .HasOne(e => e.Header)
    .WithOne(e => e.Blog)
    .OnDelete(DeleteBehavior.Restrict);  // 👈
```

Many-to-many

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Introduction

- One-to-many
- One-to-one
- **Many-to-many**

- EF Core builds **join entity type** automatically!
- Alternatively, build one yourself

Basic

- One-to-many
- One-to-one
- **Many-to-many**

- Both entity types have collection navigations for the relationship
- Mapped by convention
- Join table named `PostTag` by convention

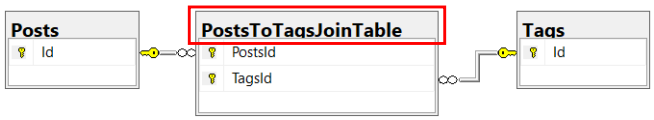
```
public class Post
{
    public int Id { get; set; }
    public List<Tag> Tags { get; } = [];
}
```

```
public class Tag
{
    public int Id { get; set; }
    public List<Post> Posts { get; } = [];
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Custom table name

- One-to-many
- One-to-one
- Many-to-many



Give an explicit name with `UsingEntity()`

Fluent API

```
modelBuilder.Entity<Post>()
  .HasMany(e => e.Tags)
  .WithMany(e => e.Posts)
  .UsingEntity("PostsToTagsJoinTable");
```

Custom foreign key names

- One-to-many
- One-to-one
- Many-to-many

Fluent API

```
modelBuilder.Entity<Post>()
  .HasMany(p => p.Tags)
  .WithMany(t => t.Posts)
  .UsingEntity
  (
    "PostsToTagsJoinTable",
    l => l.HasOne(typeof(Tag)).WithMany().HasForeignKey("PostForeignKey"),
    r => r.HasOne(typeof(Post)).WithMany().HasForeignKey("TagForeignKey")
  );
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Explicit Join Entity Type

- One-to-many
- One-to-one
- Many-to-many

- Dedicated class mapped to the join table allows easy referencing:

```
public class PostTag
{
    public int PostId { get; set; }
    public int TagId { get; set; }
}
```

Fluent API

```
modelBuilder.Entity<Post>()
    .HasMany(e => e.Tags)
    .WithMany(e => e.Posts)
    .UsingEntity<PostTag>();
```

Payload & Navigations

- One-to-many
- One-to-one
- Many-to-many

```
public class Post
{
    public int Id { get; set; }
    public List<Tag> Tags { get; } = [];
    public List<PostTag> PostTags { get; } = [];
}
```

```
public class Tag
{
    public int Id { get; set; }
    public List<Post> Posts { get; } = [];
    public List<PostTag> PostTags { get; } = [];
}
```

```
public class PostTag
{
    public int PostId { get; set; }
    public int TagId { get; set; }
    public DateTime CreatedOn { get; set; }    // 👈 payload
    public Post Post { get; set; } = null!;    // 👈 extra navigation
    public Tag Tag { get; set; } = null!;      // 👈 extra navigation
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Unidirectional relationship

- One-to-many
- One-to-one
- Many-to-many

- Not mandatory to include navigation on both sides of the relationship

```
public class Post
{
    public int Id { get; set; }
    public List<Tag> Tags { get; } = [];
}
```

```
public class Tag
{
    public int Id { get; set; }
    // -- no navigation property --
}
```

- **Not** discovered by convention, configure manually:

Fluent API

```
modelBuilder.Entity<Post>()
    .HasMany(e => e.Tags)
    .WithMany(); //no argument
```

No Cascade Delete

- One-to-many
- One-to-one
- Many-to-many

- Default cascading delete behavior can be changed:

Fluent API

```
modelBuilder.Entity<Post>()
 .HasMany(e => e.Tags)
 .WithMany(e => e.Posts)
 .UsingEntity(
   l => l.HasOne(typeof(Tag)).WithMany().OnDelete(DeleteBehavior.Restrict),
   r => r.HasOne(typeof(Post)).WithMany().OnDelete(DeleteBehavior.Restrict));
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Alternative

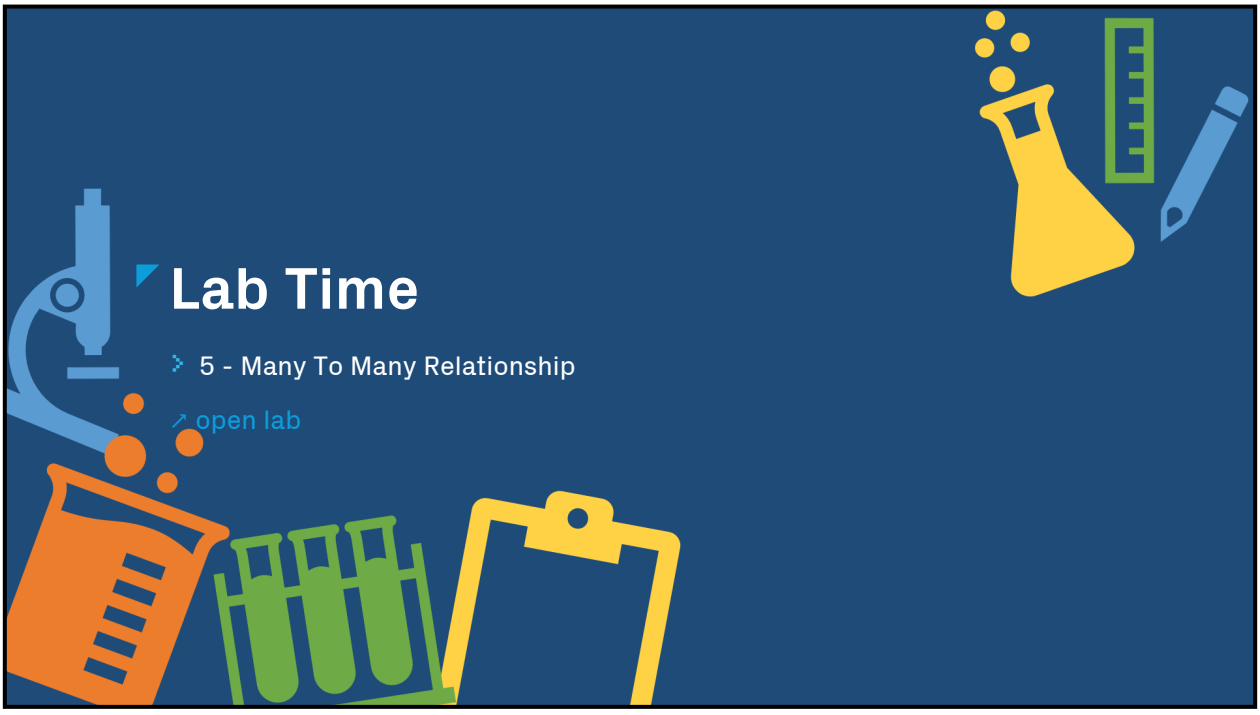
- One-to-many
- One-to-one
- Many-to-many

- Map join table to a regular entity type with two one-to-many relationships
- No special mapping is needed

```
public class Post
{
    public int Id { get; set; }
    public List<PostTag> PostTags { get; } = [];
}
```

```
public class Tag
{
    public int Id { get; set; }
    public List<PostTag> PostTags { get; } = [];
}
```

```
public class PostTag
{
    public int PostId { get; set; }
    public int TagId { get; set; }
    public Post Post { get; set; } = null!;
    public Tag Tag { get; set; } = null!;
}
```



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

1. One-to-many

- Detected by using navigation properties & foreign keys
- Required vs optional relationships
- Shadow foreign key = no foreign key defined in model
  - In One-to-many: no need to define foreign key via Fluent API!
- Unidirectional relationship discovered automatically
- FluentAPI:
  - improves readability
  - unrecognized foreign key names in model
  - no navigation properties at all
  - exception instead of cascade delete
  - self-referencing relation

Key Takeaways

2. One-to-one

- Quite similar to One-to-many, but:
- FluentAPI:
  - Pk-to-PK relationship
  - Shadow foreign key
- Unidirectional relationship:
  - No navigation in principal:
    foreign key required or FluentAPI
  - No navigation in dependent:
    discovered as one-to-many, use FluentAPI

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

3. Many-to-many

- Implicit join entity type by convention
- Explicit join entity type for more control
- FluentAPI:
  - Configure join table name and foreign keys
  - Add payload properties to join entity
- Unidirectional relationship:
  - Requires FluentAPI
  - Skip navigations or navigations to join entity
- Direct use of join table as alternative

Querying Data

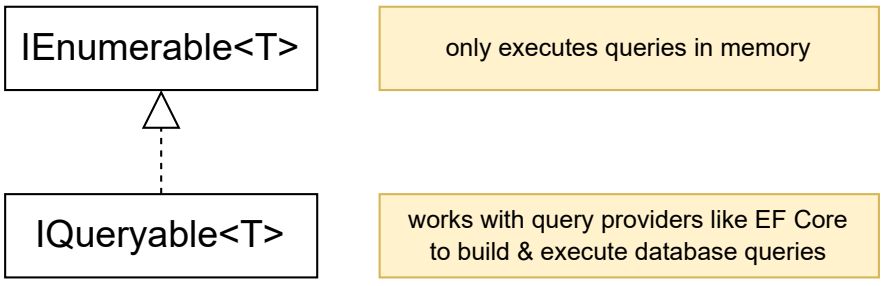
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Querying data

- EF Core uses **LINQ** to query databases
- Write strongly typed queries in C# 😎

```
IQueryable<Blog> query = context.Blogs.Where(b => b.Url.Contains("dotnet"));
```

IQueryable<T>



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

IQueryable<T>

- Represents an expression tree
- LINQ provider (EF Core) translates the expression tree into SQL

```
// LINQ query in C#
IQueryable<Blog> query = context.Blogs
  .Where(b => b.Url.Contains("dotnet"));
```

```
-- SQL Query
SELECT * FROM Blogs
WHERE Url LIKE '%dotnet%';
```

- SQL query is executed when `IQueryable` is being enumerated

LINQ Syntax

- Query Syntax
  - starts with `from`
  - ends with `select` or `group`

```
var query =
    from blog in context.Blogs
    let urlLength = blog.Url.Length
    where urlLength > 10
    orderby urlLength descending
    select new
    {
      BlogName = blog.Name,
      UrlLength = urlLength
    };
```

- Method Syntax

```
var query = context.Blogs
    .Select(blog => new
    {
      blog,
      UrlLength = blog.Url.Length
    })
    .Where(b => b.UrlLength > 10)
    .OrderByDescending(b => b.UrlLength)
    .Select(b => new
    {
      BlogName = b.blog.Name,
      UrlLength = b.UrlLength
    });
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Loading a single entity

- `Find` instead of `FirstOrDefault`

```
using (var context = new BloggingContext();

Blog? blog = await context.Blogs.FirstOrDefaultAsync(b => b.BlogId == 1);
Blog? blog = await context.Blogs.FindAsync(1);  // more elegant way (EF Core)
```

Loading a single entity

- Look-alikes:

Method	If no match
`First()`	throws error
`FirstOrDefault()`	returns `null` (for reference type) or default value (for value type)
`Single()`	throws error (also when more than one match)
`SingleOrDefault()`	returns `null` (for reference type) or default value (for value type) throws error when more than one match

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Evaluation on Server or Client

Server & Client evaluation

- EF Core attempts to evaluate a query on the db server as much as possible
- Query parts may be evaluated on the client

```
var blogs = await context.Blogs
    .Where(blog => blog.Rating > 3)
    .Select(blog => new { blog.BlogId, Url = StandardizeUrl(blog.Url) })
    .ToListAsync();
```

? Which parts run on the server and which on the client?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Unsupported client evaluation

If EF Core can't translate a query part to the server,
a runtime exception is thrown

```
var blogs = await context.Blogs
    .Where(blog => StandardizeUrl(blog.Url).Contains("dotnet"))
    //             ^^^^^^^^^^^^^^^^^^^^^^^^^^
    .ToListAsync();
```

Explicit client evaluation

Explicitly opt into client evaluation using `AsEnumerable` or `ToList`:

- `AsEnumerable`: streams results
- `ToList`: executes query & loads all results into a list

```
var blogs = await context.Blogs
    .AsEnumerable()
    .Where(blog => StandardizeUrl(blog.Url).Contains("dotnet"))
    .ToListAsync();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Tracking vs. No-Tracking Queries

Change Tracker

- EF Core's **change tracker** monitors changes made to entity instances
- Ensures correct SQL commands to update the database: `INSERT`, `UPDATE`, `DELETE`
- LINQ queries can be **tracking** or **no-tracking**
- More on the change tracker later ...

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Tracking queries

By default, queries that return entity types are tracking:

```
var blog = await context.Blogs.FindAsync(1);  // 👈 tracking

blog.Rating = 5;
await context.SaveChangesAsync();             // Persist changes to the db
```

No-tracking queries

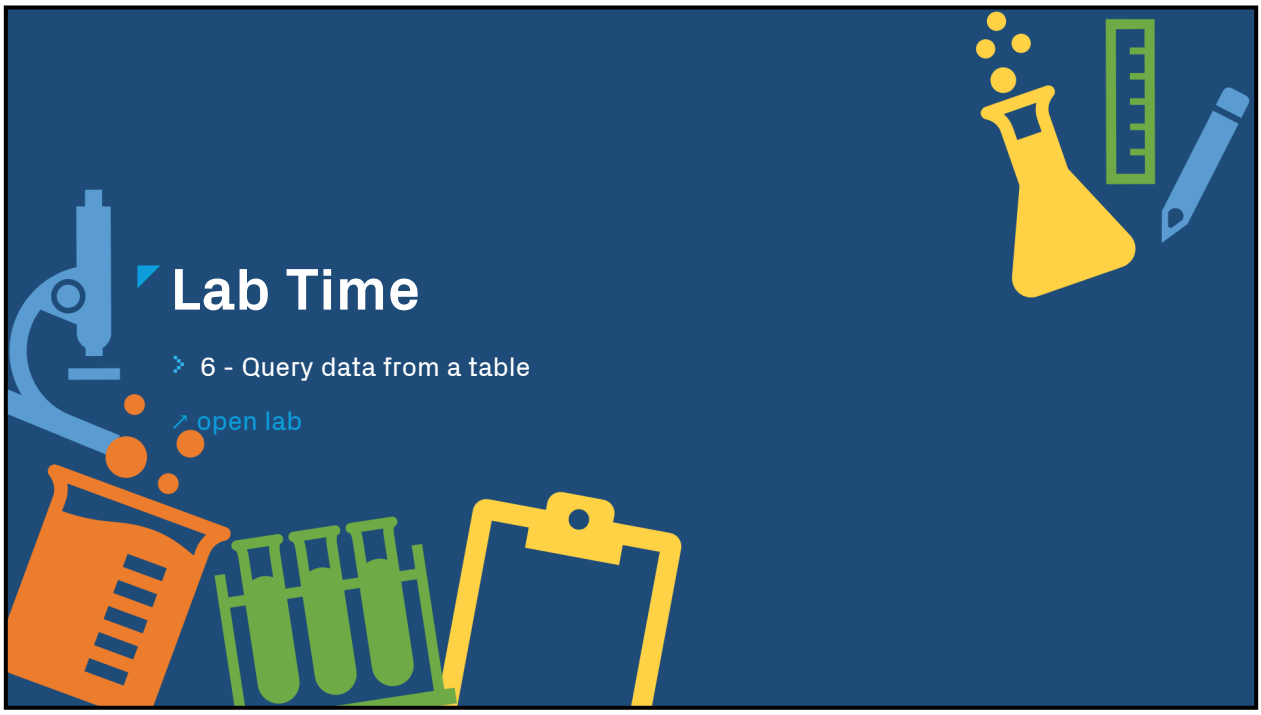
- Useful in read-only scenarios
- Faster 🚀: no need to set up the change tracking information
- Fetch blogs for display purposes only:

```
var blogs = await context.Blogs
    .AsNoTracking()   // 👈 no-tracking
    .ToListAsync();
```

- Optionally, change default tracking behavior at the context instance level:

```
context.ChangeTracker.QueryTrackingBehavior = QueryTrackingBehavior.NoTracking;
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Three ways

- Eager
- Explicit
- Lazy

Load related data using navigation properties:

- **Eager loading** - with the initial query
- **Explicit loading** - at a later time, on demand
- **Lazy loading** - automatically when navigation property is accessed

`Include`

- **Eager**
- Explicit
- Lazy

Load specified navigation properties as part of the main query:

```
using var context = new BloggingContext();

var blogs = await context.Blogs
    .Include(blog => blog.Posts)    // 👆
    .Include(blog => blog.Owner)    // 👆
    .ToListAsync();
```

- Load related entities in the same SQL query
- No extra round-trips

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

ThenInclude

- Eager
- Explicit
- Lazy

- Drill down through relationships to include multiple levels of related data

```
using var context = new BloggingContext()

var blogs = await context.Blogs
    .Include(blog => blog.Posts)
    .ThenInclude(post => post.Author)    // 👈
    .ToListAsync();
```

Combine

- Eager
- Explicit
- Lazy

- Combine multiple levels and roots in one query:

```
using var context = new BloggingContext();

var blogs = await context.Blogs
    .Include(blog => blog.Posts)
    .ThenInclude(post => post.Author)
    .ThenInclude(author => author.Photo)     // 👈 Chaining ThenInclude
    .Include(blog => blog.Owner)
    .ThenInclude(owner => owner.Photo)
    .Include(blog => blog.Posts)             // 👈 "Restart": Load Posts again
    .ThenInclude(post => post.Tags)
    .ToListAsync();
```

- EF will combine the joins when generating SQL

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Navigation chains

- Must consist of **references**, only final link may be a **collection**
- 2 collections after eachother would result in an explosion of data!

```
.Include(blog => blog.Owner.AuthoredPosts)
//                    ref       coll          ✅

.Include(post => post.Blog.Owner.Photo)
//                    ref  ref   ref          ✅

.Include(blog.Owner.AuthoredPosts.Comments)
//             ref       coll       coll       ❌
```

Filtered `Include`

- Eager
- Explicit
- Lazy

```
using var context = new BloggingContext();

var filteredBlogs = await context.Blogs
    .Include(
        blog => blog.Posts
            .Where(post => post.BlogId == 1)
            .OrderByDescending(post => post.Title)
            .Take(5)
    )
    .ToListAsync();
```

- Supported operations:
  - `Where`
  - `OrderBy`
  - `OrderByDescending`
  - `ThenBy`
  - `ThenByDescending`
  - `Skip`
  - `Take`

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Include on derived types

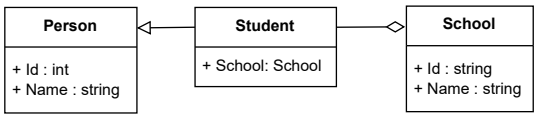
- Eager
- Explicit
- Lazy

Given the following schema:

```
public class Person
{
  public int Id { get; set; }
  public string Name { get; set; }
}
```

```
public class Student : Person
{
  public School School { get; set; }
}
```

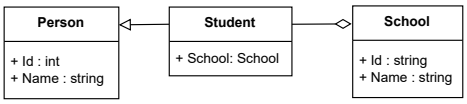
```
public class School
{
  public int Id { get; set; }
  public string Name { get; set; }
  public List<Student> Students { get; set; }
}
```




```
public class SchoolContext : DbContext
{
  public DbSet<Person> People { get; set; }
  public DbSet<School> Schools { get; set; }

  protected override void OnModelCreating(ModelBuilder builder) {
    builder.Entity<School>()
      .HasMany(s => s.Students)
      .WithOne(s => s.School);
  }
}
```

Include on derived types

- Eager
- Explicit
- Lazy



How to include `School` when retrieving `People`?

```
// 1. use cast
context.People.Include(person => ((Student)person).School).ToList();

// 2. use `as` operator
context.People.Include(person => (person as Student).School).ToList();

// 3. use overload of `Include`
context.People.Include("School").ToList();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

`AutoInclude` Navigation

- Eager
- Explicit
- Lazy

Configure a navigation to always load with the entity

Define

Fluent API

```
modelBuilder.Entity<Theme>()
  .Navigation(e => e.ColorScheme)
  .AutoInclude();
```

Use

```
using var context = new BloggingContext();

// Include ColorScheme automatically:
var themes = await context.Themes
                .ToListAsync();

// Ignore previously configured AutoInclude:
var themes = await context.Themes
                .IgnoreAutoIncludes()
                .ToListAsync();
```

Explicit Loading

- Eager
- Explicit
- Lazy

- Load specified navigation properties *explicitly*
- Use the `DbContext.Entry` API

```
using var context = new BloggingContext();

var blog = await context.Blogs.SingleAsync(b => b.BlogId == 1);

await context.Entry(blog)
  .Collection(b => b.Posts)    // 👆 0 or more Posts
  .LoadAsync();

await context.Entry(blog)
  .Reference(b => b.Owner)     // 👆 one Owner
  .LoadAsync();
```

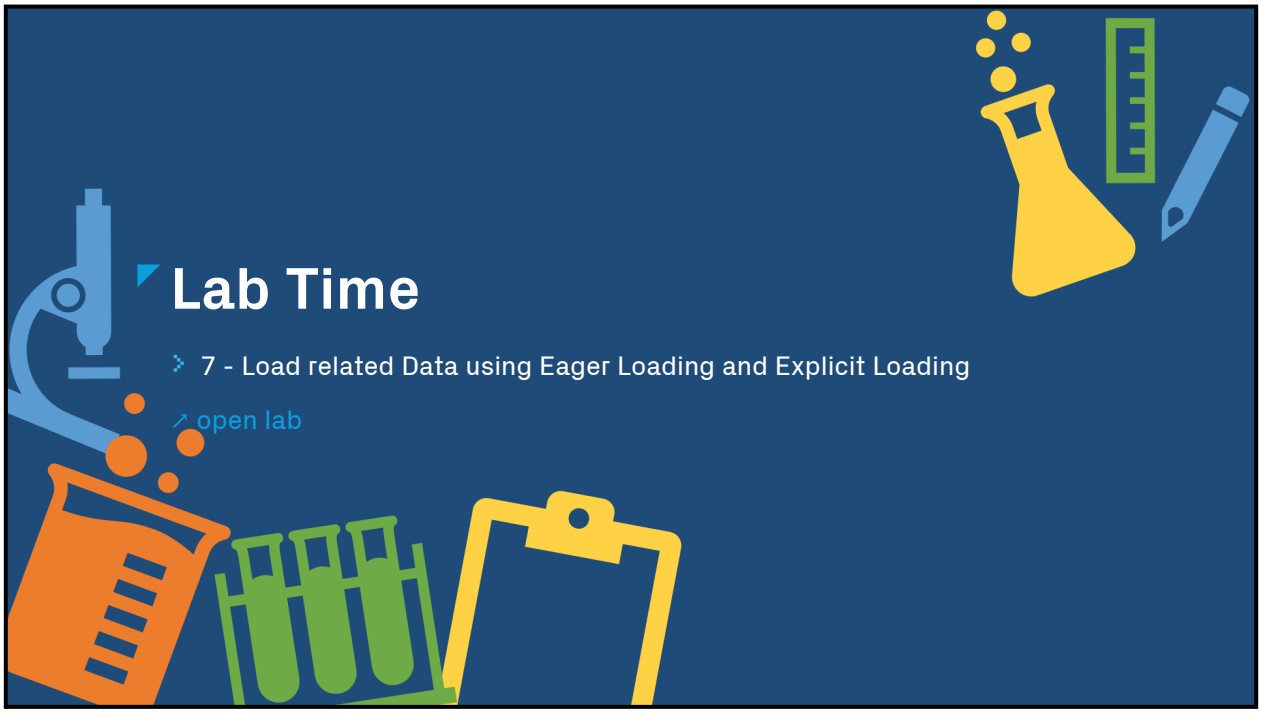
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Query Navigation

- Eager
- Explicit
- Lazy

- `Query()` returns `IQueryable`
- Avoid loading all related rows
- Run operations database-side: `Count` , `Where`

```
using var context = new BloggingContext();

var blog = await context.Blogs
    .SingleAsync(b => b.BlogId == 1);

var goodPosts = await context.Entry(blog)
    .Collection(b => b.Posts)
    .Query()                    // 👆 do not load all posts into memory
    .Where(p => p.Rating > 3)   // 👆
    .ToListAsync();
```



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lazy Loading

- Eager
- Explicit
- **Lazy**

- Navigational properties queried automatically when needed
- Two main implementation strategies:
  - using proxies
  - injecting LazyLoader
- Can cause unneeded extra database roundtrips: *N+1 problem*

Using Proxies

- Eager
- Explicit
- **Lazy**

3 requirements:

1. Install Nuget package `Microsoft.EntityFrameworkCore.Proxies`
2. Ensure *all* navigation properties are *overridable*, so:
   property is `virtual` and class is not sealed

```
public class Blog
{
    public int Id { get; set; }
    public string Name { get; set; }

    public virtual ICollection<Post> Posts { get; set; }
}
```

```
public class Post
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }

    public virtual Blog Blog { get; set; }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Using Proxies

- Eager
- Explicit
- **Lazy**

3. Call `.UseLazyLoadingProxies()`

```
var contextOptions = new DbContextOptionsBuilder<AppDbContext>()
    .UseLazyLoadingProxies()          // 👈
    .UseSqlServer(myConnectionString)
    .Options;
```

Injecting LazyLoader Service

- Eager
- Explicit
- **Lazy**

3 requirements:

1. Install Nuget package `Microsoft.EntityFrameworkCore.Abstractions`

```
public class Blog {
  private ILazyLoader LazyLoader { get; set; } = null!;

  public Blog() {}
  private Blog(ILazyLoader lazyLoader) {                // 2️⃣ Inject LazyLoader service
    LazyLoader = lazyLoader;
  }

  // Other properties

  public ICollection<Post>? Posts {
    get => LazyLoader?.Load(this, ref field) ?? field;   // 3️⃣ Load lazily when accessed
    set => field = value;
  }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Comparison

- **Eager**
  - ✅ fewer DB trips, immediate data access
  - ❌ complex queries, higher memory use
- **Explicit**
  - ✅ precise control when, avoids unnecessary data
  - ❌ N+1 query problem, more DB trips, verbose code
- **Lazy**
  - ✅ loads only when needed, reduces initial query size
  - ❌ N+1 query problem, hidden performance issues

When to use which? 🤔

1. Related data is rarely accessed or only needed on demand
2. All related data is needed immediately and DB round trips must be minimized
3. Specific related data is conditionally required

> ❓ Map **Explicit**, **Eager** and **Explicit**

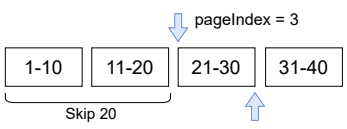
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Pagination

Pagination

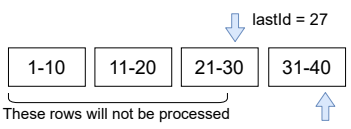
- Used for large resultsets in combination with a user interface
- Offset pagination

```
int pageSize = 10;
int pageIndex = 3;
int position = (pageIndex - 1) * pageSize;
var nextPage = await context.Posts
    .OrderBy(b => b.PostId)
    .Skip(position)   // 👆 OFFSET
    .Take(pageSize)   // 👆 FETCH
    .ToListAsync();
```



- Keyset pagination

```
int pageSize = 10;
int lastId = 27; // tracks last fetched entry ID

var nextPage = await context.Posts
    .OrderBy(b => b.PostId)
    .Where(b => b.PostId > lastId)
    .Take(pageSize)
    .ToListAsync();
```

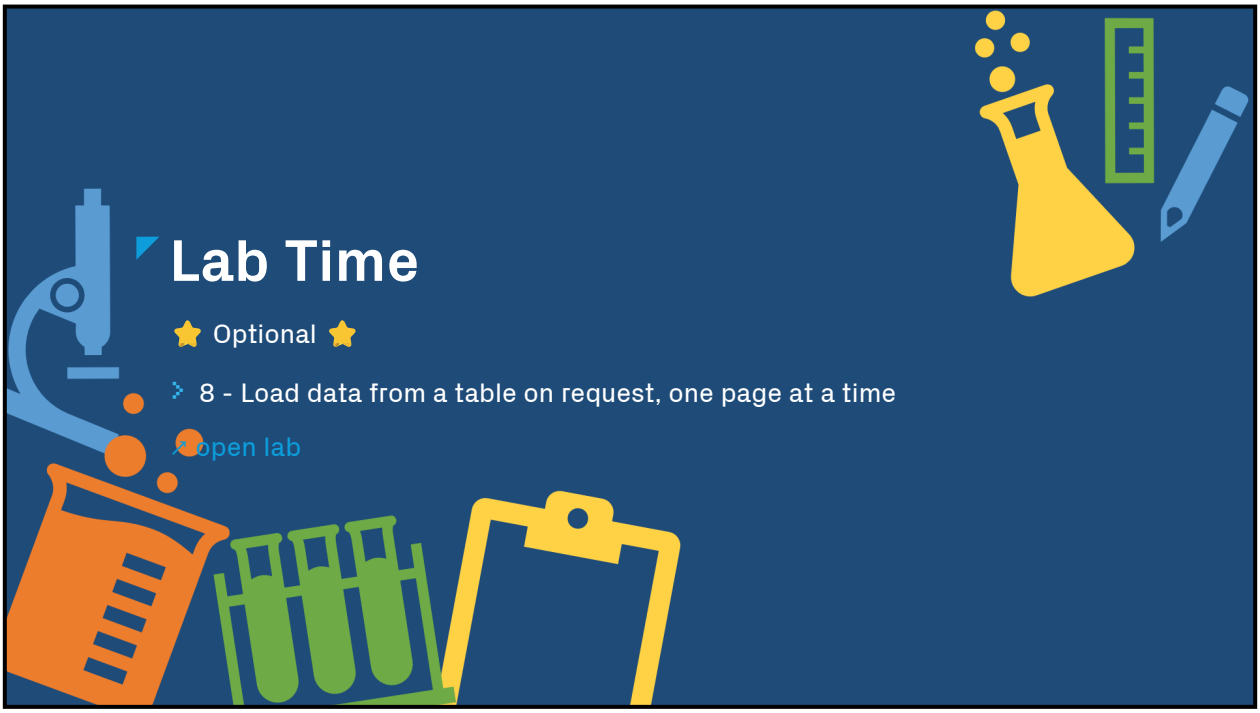


© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Pros and Cons

- Offset pagination
  `Skip` / `Take`
  - ✅ Possible to jump to a random page
  - ✅ Easier implementation
  - ❌ DB must still process 'skipped' entries
  - ❌ Possible skipped or duplicate entries
- Keyset pagination
  `Where` / `Take`
  - ✅ Better performance
  - ✅ Consistence results
  - ❌ No random access
  - ❌ Records must be stored sequentially
  - ❌ Client needs to manage the keyset value

? Which navigation to use for jumping to a specific page, and which for navigating to previous/next page?



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

- Loading single entity: `FindAsync(id)`
- EF Core prefers query evaluation on db server
  - Enforce client evaluation: `.AsEnumerable()` in LINQ query
- `.AsNoTracking()` for read-only scenario's
- Load related data:
  - Eager: `Include()` and `ThenInclude()`
  - Explicit: `.Collection().LoadAsync()` and `.Reference().LoadAsync()`
  - Lazy: use proxies or inject `ILazyLoader`
- Pagination:
  - Offset: jump to random page
  - Keyset: better performance & ideal for previous/next

CRUD Operations

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Change data in database

INSERT, UPDATE or DELETE records

Two main approaches:

1. `SaveChanges` - track and persist changes
2. `ExecuteUpdate` / `ExecuteDelete` - bulk operations

SaveChanges


```
using var context = new BloggingContext();

var blog = context.Blogs.Single(b => b.Title == "EF Core");  // 1 Load
blog.Title = "EF Core Advanced";                             // 2 Modify
context.SaveChanges();                                       // 3 Persist
```

1. EF Core tracks the loaded entity in its **change tracker**
2. EF Core is not involved
3. EF Core persists detected changes: `UPDATE` SQL command

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Flexible

- Automatically tracks changes to entities and properties
- Batches multiple changes to reduce db roundtrips
- Detects concurrency issues
- Wraps changes in a transaction automatically

Less performant

- Not suitable for large datasets:

```
using var context = new BloggingContext();

var blogs = context.Blogs
  .Where(b => b.Rating < 3)
  .ToList();

foreach (var blog in blogs)
{
  context.Blog.Remove(blog);
}

context.SaveChanges();
```

```
DELETE FROM Blog WHERE Id = 1;
DELETE FROM Blog WHERE Id = 2;
DELETE FROM Blog WHERE Id = 3;
DELETE FROM Blog WHERE Id = 4;
DELETE FROM Blog WHERE Id = 5;
DELETE FROM Blog WHERE Id = 6;

-- ... etc, etc
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Bulk operations

Use `ExecuteUpdate` or `ExecuteDelete` for bulk update scenarios

- Very efficient database execution
- Only one `UPDATE` or `DELETE`:

```
using var context = new BloggingContext();

context.Blogs
  .Where(b => b.Rating < 3)
  .ExecuteDelete();   // 👆
```

```
DELETE FROM Blog WHERE Rating < 3;
```

- Other advantages/disadvantages will be discussed

Basic SaveChanges

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Add

- Add new instances of your entity classes

```
using var context = new BloggingContext();

var blog = new Blog { Url = "http://example.com" };

context.Blogs.Add(blog);
context.SaveChanges();
```

- `SaveChanges` - insert data into the db

AddRange

- Add multiple new instances in a single call

```
using var context = new BloggingContext();

List<Blog> blogs = [
    new Blog { Url = "http://example.com/blog1" },
    new Blog { Url = "http://example.com/blog2" }
];

context.Blogs.AddRange(blogs);
context.SaveChanges();
```

- `SaveChanges` - insert all data into the db in *one transaction*

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Update

EF Core automatically tracks changes to entities:

- queried or loaded from the db
- previously added and saved to the db

```
using var context = new BloggingContext()

var blog = context.Blogs.Single(b => b.Url == "http://example.com");

blog.Url = "http://example.com/blog";
context.SaveChanges();
```

Remove

- Delete instances of your entity classes

```
using var context = new BloggingContext();

var blog = context.Blogs.Find(12);

context.Blogs.Remove(blog);   // 👈
context.SaveChanges();
```

- `SaveChanges`:
  - deletes entities already in the db
  - unsaved *added* entities are simply removed from the context
  - `RemoveRange()` exists as well

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Multiple operations

```
using var context = new BloggingContext();

context.Blogs.Add(new Blog { Title = "JavaScript" });          // +
context.Blogs.Add(new Blog { Title = "TypeScript" });          // +

var firstBlog = context.Blogs.First();
firstBlog.Title = "";                                          // ✎

var lastBlog = context.Blogs.OrderBy(e => e.BlogId).Last();
context.Blogs.Remove(lastBlog);                                // −

context.SaveChanges();
```

- `SaveChanges()` starts a transaction automatically
  - If any operation fails, the transaction rolls back - maintaining consistency

ChangeTracker

- Each **DbContext instance** tracks changes to entities
- `SaveChanges()` generates SQL commands based on tracked changes

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

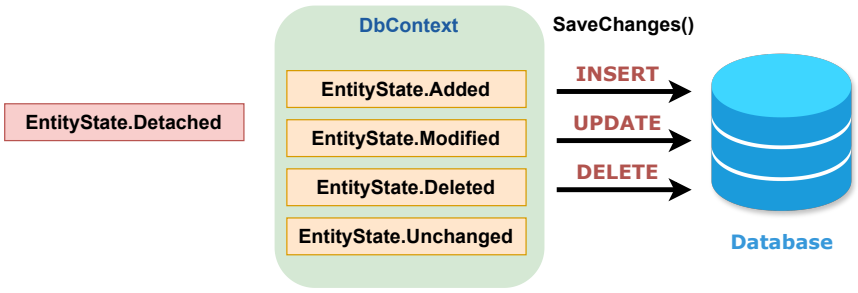
Entity states

Each entity has an associated `EntityState`

- **Detached**: Not tracked by the `DbContext`
- **Added**: New entity, not yet in the database
- **Unchanged**: No changes since queried
- **Modified**: Properties changed since queried
- **Deleted**: Marked for deletion from the database

Which will cause `SaveChanges()` to generate database commands?

Entity States



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Query & Update

```
using var context = new BlogsContext();

var blog = context.Blogs
    .Include(e => e.Posts)
    .First(e => e.Name == ".NET Blog");

blog.Name = ".NET Blog (Updated!)";

foreach (var post in blog.Posts
    .Where(e => !e.Title.Contains("5.0")))
{
    post.Title = post.Title.Replace("5", "5.0");
}

context.SaveChanges();
```

```
UPDATE "Blogs"
SET "Name" = @p0
WHERE "Id" = @p1;

UPDATE "Posts"
SET "Title" = @p0
WHERE "Id" = @p1;
```

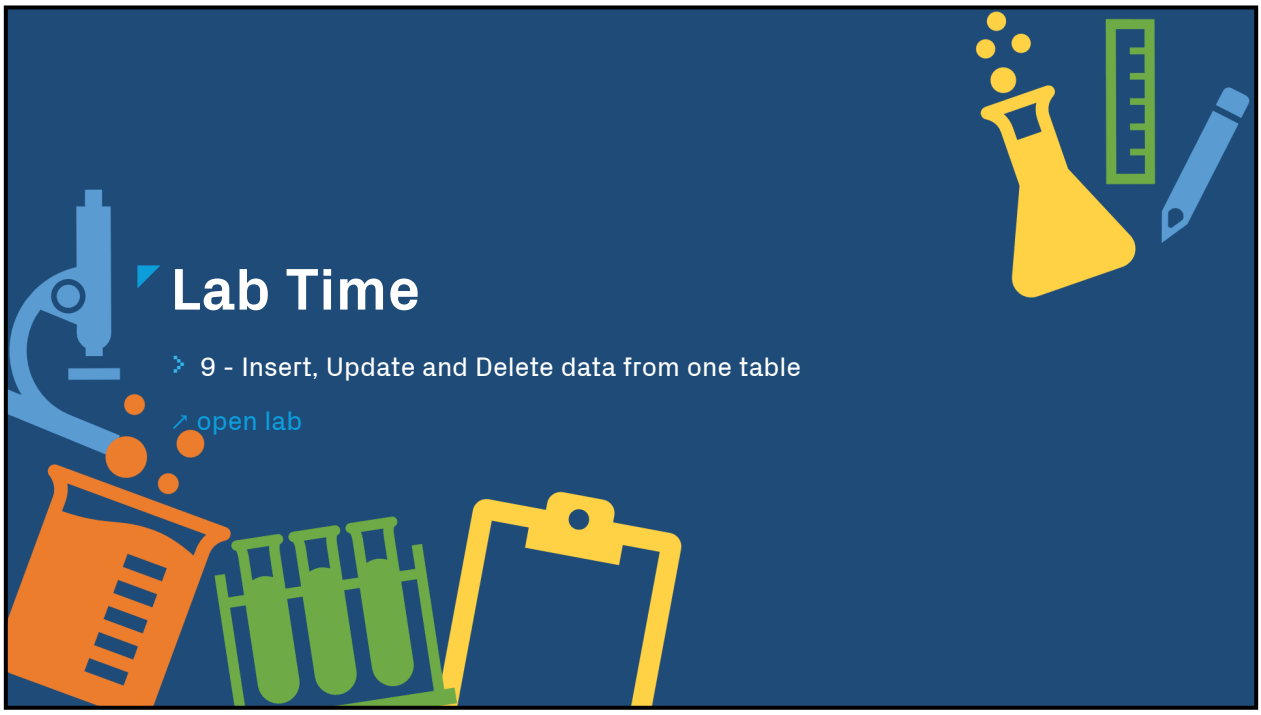
Debug View

Visualize tracked entities and their states

```
// ... load, add, modify, delete entities ...

context.ChangeTracker.DetectChanges();                          // 👆
Console.WriteLine(context.ChangeTracker.DebugView.LongView);    // 👆

context.SaveChanges()
```

```
Blog {Id: 1} Modified
  Id: 1 PK
  Name: '.NET Blog (Updated!)' Modified Originally '.NET Blog'
  Posts: [{Id: 1}, {Id: 2}, {Id: 3}]
Post {Id: 1} Unchanged
  Id: 1 PK
  BlogId: 1 FK
  Content: 'Announcing the release of EF Core 5.0, a full featured cross...'
  ...
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Add related entity

- Relationships tracked automatically

```
using var context = new BloggingContext();

var blog = new Blog
{
    Url = "http://blogs.msdn.com/dotnet",
    Posts = new List<Post>
    {
        new Post { Title = "Intro to C#" },
        new Post { Title = "Intro to VB.NET" },
    }
};

context.Blogs.Add(blog);
context.SaveChanges();  // 👈 blog + 2 posts added
```

Change relationship

- Updating a navigation property updates its foreign key automatically

```
using var context = new BloggingContext();

var blog = new Blog { Url = "http://blogs.msdn.com/visualstudio" };
var post = context.Posts.First();

post.Blog = blog;
context.SaveChanges();   // 👈 foreign key updated automatically
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Remove relationship

```
using var context = new BloggingContext();

var blog = context.Blogs.Include(b => b.Posts).First();
var post = blog.Posts.First();

blog.Posts.Remove(post);
context.SaveChanges(); // 👈
```

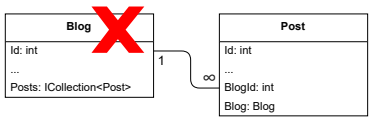
- Impact depends on type of relationship: *optional* or *required*
- More on this in the next module

Cascade Deletes

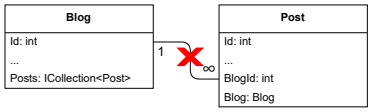
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Orphans

- A child left without parent is an 'orphan' - occurs in **two scenarios**:



Parent is deleted



Child is no longer linked to it

Parent Deleted

Deleting a blog:

- **required relationship** -> cascade delete related posts
- **optional relationship** -> set foreign keys to `null`

```
using var context = new BlogsContext();

var firstBlog = context.Blogs.OrderBy(e => e.Name).Include(e => e.Posts).First();

context.Remove(firstBlog);   // 👈
context.SaveChanges();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Link Removed

Same behavior:

- **required** -> cascade delete
- **optional** -> set foreign keys to `null`

```
using var context = new BlogsContext();

var firstBlog = context.Blogs.OrderBy(e => e.Name).Include(e => e.Posts).First();

firstBlog.Posts.Clear();   // 👈
context.SaveChanges();
```

Who enforces?

Cascade deletes can be enforced by:

1. **EF Core** (client-side)

```
// Posts ARE tracked - EF Core handles cascade
var firstBlog = context.Blogs.Include(e => e.Posts).First();
```

2. **Database** (server-side)

```
// Posts NOT tracked - Database handles cascade (if configured)
var firstBlog = context.Blogs.First();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

🔧 Configure Delete Behavior

- EF Core
- Database

- Set the delete behavior per relationship:

Fluent API

```
modelBuilder
    .Entity<Blog>()
    .HasOne(e => e.Owner)
    .WithOne(e => e.OwnedBlog)
    .OnDelete(DeleteBehavior.Cascade);    // 👈
```

1. Configures database schema when created by EF Core
2. Defines how EF Core treats **tracked** entities

DeleteBehavior Options

DeleteBehavior	DB Schema	EF Core Behavior
Cascade ⭐	ON DELETE CASCADE	Deletes children
ClientSetNull ⭐	No constraint	Sets FK to null
SetNull	ON DELETE SET NULL	Sets FK to null
ClientCascade	No constraint	Deletes children
NoAction	ON DELETE NO ACTION	Sets FK to null (optional relationship only)
Restrict	ON DELETE RESTRICT	Sets FK to null (optional relationship only)
ClientNoAction	No constraint	No action (throws)

⭐ = Recommended choices

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Best Practices

Required Relationships:

```
.OnDelete(DeleteBehavior.Cascade)          // Default - recommended
```

- Children deleted automatically
- Works even if children not loaded (via DB Cascade)

Optional Relationships:

```
.OnDelete(DeleteBehavior.ClientSetNull)    // Recommended
```

- FKs set to null by EF Core
- Fails if children not loaded (safety check)

Always load children when deleting parents to avoid surprises!

Common Pitfall

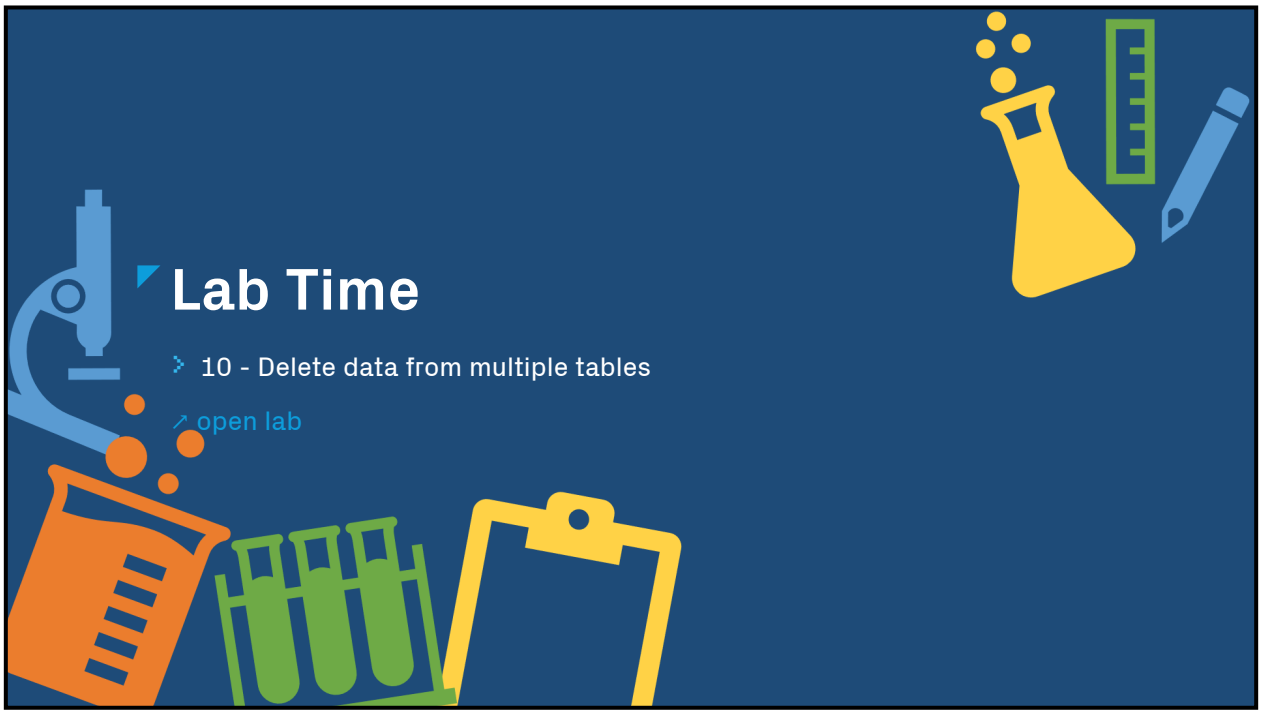
❌ DANGER: Posts not loaded

```
var firstBlog = context.Blogs.First();
context.Remove(blog);
context.SaveChanges();
// Behavior depends on DB configuration!
```

✅ SAFE: Posts loaded, EF Core handles it

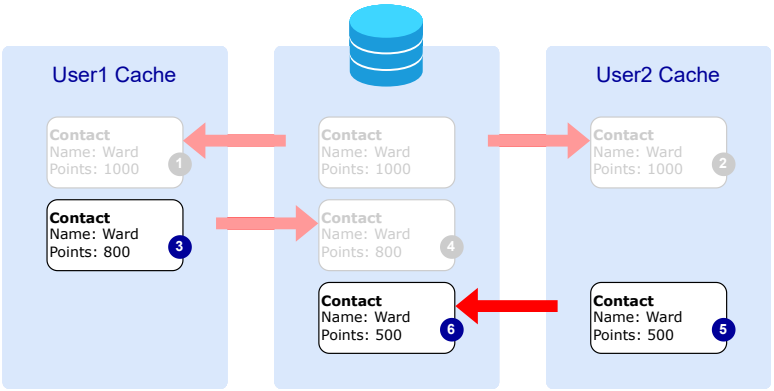
```
var firstBlog = context.Blogs.Include(e => e.Posts).First();
context.Remove(blog);
context.SaveChanges();
// EF Core enforces configured behavior
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Concurrent Data Access



Optimistic Concurrency

- Data modification will fail on save if data has changed since it was queried
- Default in EF Core
- Assumes concurrency conflicts are relatively rare
- No data locking during updates
- Failure is reported to the application which can handle accordingly

? How can EF Core detect that data has changed in the meantime?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Concurrency Token

- Entity property configured as a **concurrency token**
- Loaded and tracked when entity is queried
- When UPDATE or DELETE during `SaveChanges()`:
  compare **database value** with **original value** read by EF Core

🔧 Database-managed token

- A `rowversion`/`timestamp`/`xmin` column stores a concurrency token
- Automatically updated by the database whenever the row is modified
- Supported by several database systems

Fluent API

```
modelBuilder.Entity<Person>()
    .Property(p => p.Version)
    .IsRowVersion();
```

Data annotation

```
[Timestamp]
public byte[] Version { get; set; }
```

```
var person = context.People
              .Single(b => b.FirstName == "John");

person.FirstName = "Paul";
context.SaveChanges();
```

```
UPDATE People
SET FirstName = @p0
WHERE PersonId = @p1
AND Version = @p2;     -- 👈
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

🔧 Use Entity property as concurrency token

- Allow optimistic concurrency on databases where no native automatically-updating type exists, like in SQLite:

Fluent API

```
modelBuilder.Entity<Person>()
    .Property(p => p.Salary)
    .IsConcurrencyToken();
```

Data annotation

```
public class Person
{
    [ConcurrencyCheck]
    public decimal Salary { get; set; }
}
```

```
var person = context.People.Single(b => b.FirstName == "John");
person.Salary *= 1.1m;
context.SaveChanges();              // db value compared with original value
```

Conflicts

```
var person = context.People.Find(23);
person.PhoneNumber = "555-555-5555";

// --- Another user also updates the PhoneNumber ---

try {
  context.SaveChanges();
} catch (DbUpdateConcurrencyException ex) {    // 💥 thrown by SaveChanges()

  // --- Resolve conflicts here ---

}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Resolve concurrency conflicts

Three strategies:

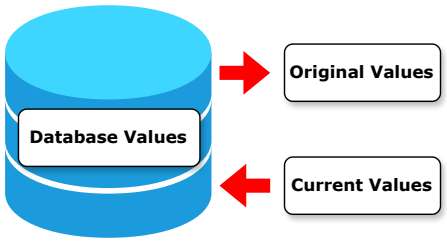
1. 🗄️ **Database Wins** - keep database values
   Inform user that their changes were discarded

2. 👩‍💻 **Client Wins** - overwrite database values
   *Simple to implement, but **risk of data loss***

3. 🔀 **Merge** - merge pending changes with new values in the database
   Process may be directed by a UI displaying both sets of values

Values

Consider these value groups when handling conflicts:



- **Original Values**
  - initially retrieved from the db
- **Database Values**
  - presently stored in the db
- **Current Values**
  - application tried to save to the db

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Database Wins

```
} catch (DbUpdateConcurrencyException ex) {
  foreach (var entry in ex.Entries) {
    if (entry.Entity is Person) {
      var databaseValues = entry.GetDatabaseValues();    // Current DB values

      entry.CurrentValues.SetValues(databaseValues);     // 👈
      Console.WriteLine("Changes discarded due to a concurrency conflict");
    }
  }
}
```

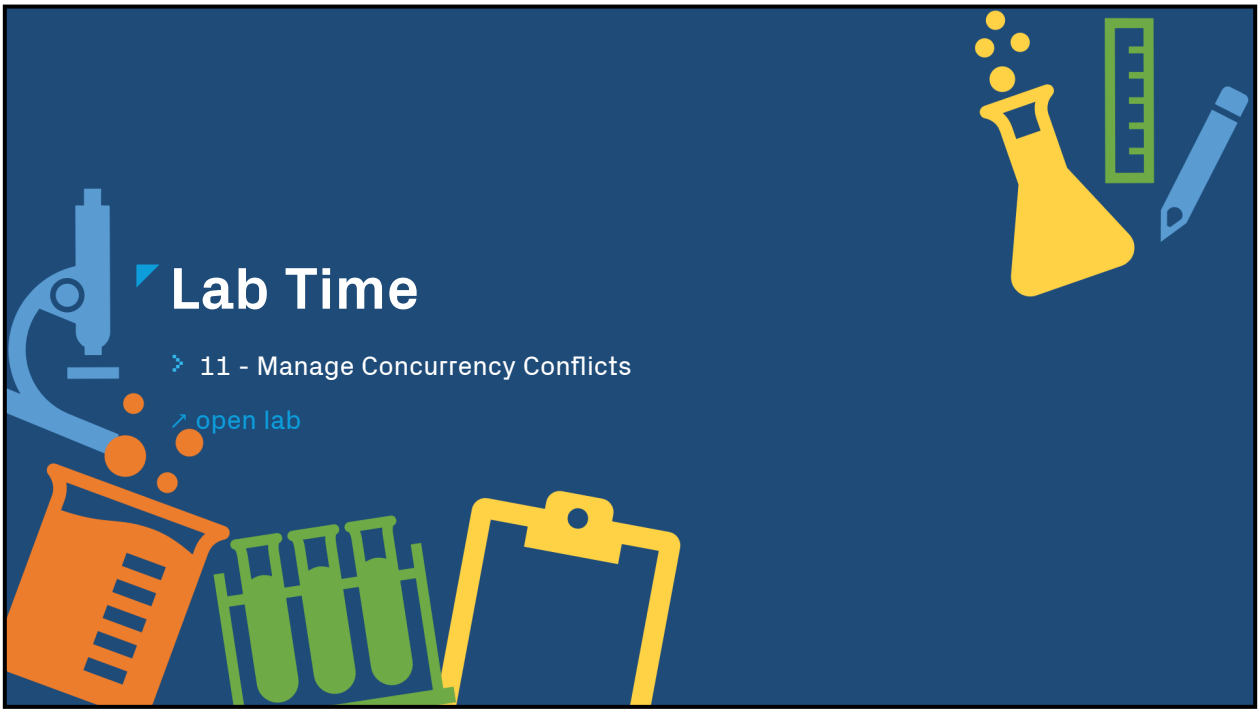
Client Wins

```
} catch (DbUpdateConcurrencyException ex) {
  foreach (var entry in ex.Entries) {
    if (entry.Entity is Person) {
      var databaseValues = entry.GetDatabaseValues();    // Current DB values

      entry.OriginalValues.SetValues(databaseValues);    // 👈
      context.SaveChanges();                             // 👈
    }
  }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Merge

```
} catch (DbUpdateConcurrencyException ex) {
  foreach (var entry in ex.Entries) {
    if (entry.Entity is Person) {
      var proposedValues = entry.CurrentValues;          // Client's changes
      var databaseValues = entry.GetDatabaseValues();    // Current DB values

      foreach (var property in proposedValues.Properties) {
        var proposedValue = proposedValues[property];
        var databaseValue = databaseValues[property];
        proposedValues[property] = databaseValue;        // Pick db or proposed value
      }
      entry.OriginalValues.SetValues(databaseValues);    // Refresh to bypass next check
    }
  }

  context.SaveChanges();     // Retry saving changes with merged values
}
```



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

ExecuteDelete and ExecuteUpdate

ExecuteDelete

- Executes very efficiently in the database
- Does not load any data from the database
- Does not involve EF's change tracker

```
context.Blogs
  .Where(b => b.Rating < 3)
  .ExecuteDelete();
```

```
DELETE
FROM Blogs AS b
WHERE b.Rating < 3
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

ExecuteUpdate

Call `SetProperty` within `ExecuteUpdate`

Update multiple properties in a single invocation

```
context.Blogs
  .Where(b => b.Rating < 3)
  .ExecuteUpdate(setters => setters
    .SetProperty(b => b.Rating, 0)
    .SetProperty(b => b.IsVisible, false)
  );
```

```
UPDATE blogs
SET Rating = 0,
    IsVisible = CAST(0 AS bit)
WHERE Rating < 3
```

How to *increase* the existing Rating with 3?

Reference related entity

- Apply `ExecuteUpdate` on projected data

```
context.Blogs
    .Select(b => new {
      Blog = b,
      NewRating = b.Posts.Average(p => p.Rating)
    })
    .ExecuteUpdate(setters =>
        setters.SetProperty(
          b => b.Blog.Rating,
          b => b.NewRating
        )
    );
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Change tracking

Don't mix `SaveChanges` and `ExecuteUpdate/ExecuteDelete`

> Assuming a start rating of 2 for *Someblog*,
> what is its rate after next code has run?

```
var blog = context.Blogs.Single(b => b.Name == "SomeBlog");

context.Blogs.ExecuteUpdate(setters =>
    setters.SetProperty(b => b.Rating, b => b.Rating + 1)
);

blog.Rating += 2;
context.SaveChanges();
```

Transactions

- ExecuteDelete
- ExecuteUpdate

They do not start a transaction implicitly like `SaveChanges` does

- Explicitly start a transaction to wrap multiple operations:

```
using (var transaction = context.Database.BeginTransaction())
{
    context.Blogs.ExecuteUpdate(/* some update */);
    context.Blogs.ExecuteUpdate(/* another update */);

    // ...
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Concurrency Control

- ExecuteDelete
- ExecuteUpdate

They return the number of rows that were affected by the operation

- Can be handy for implementing concurrency control

```
// Load the Id and ConcurrencyToken for a Blog in the database ...

var numUpdated = context.Blogs
    .Where(b => b.Id == id && b.ConcurrencyToken == concurrencyToken)
    .ExecuteUpdate(/* some update */);

if (numUpdated == 0) throw new Exception("Update failed!");
```

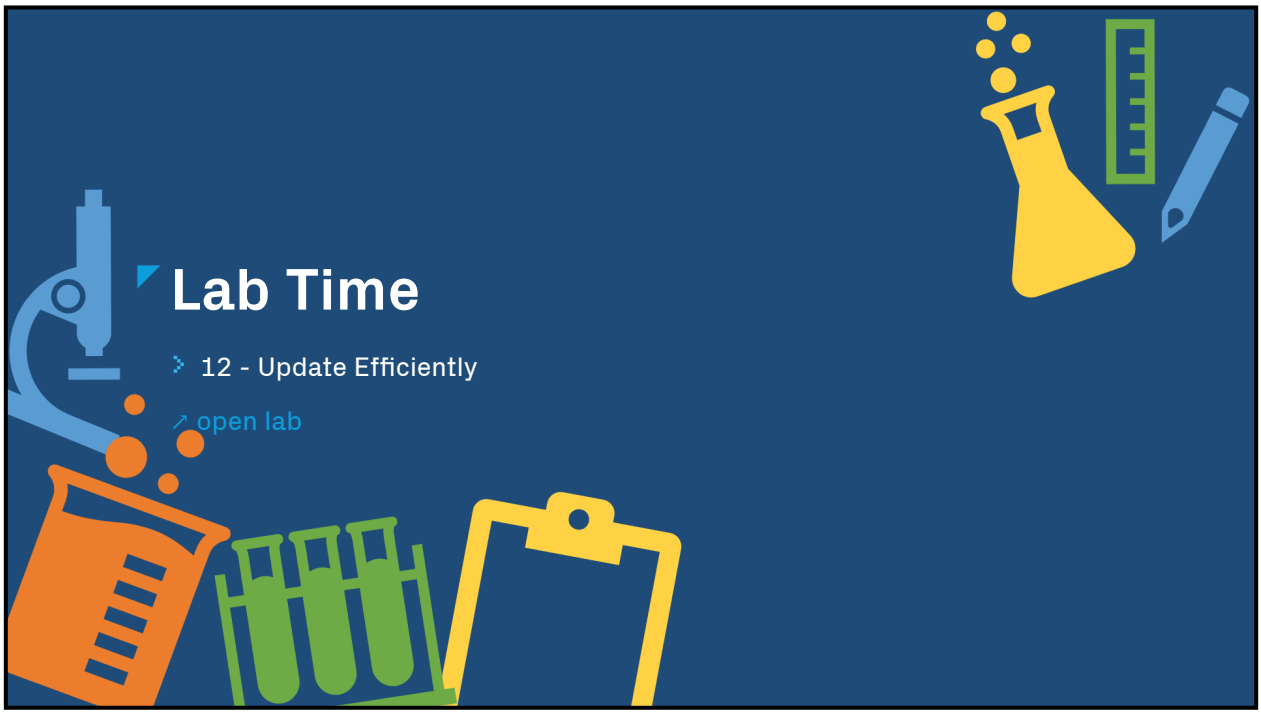
Pick best strategy 🤔

❓ Given each scenario, which strategy is best?

I want to:

1. *load a blog from the database and display a form so the user can change it*
2. *give all employees a raise*
3. *update a blog, changing some of its posts and deleting others*
4. *delete all blogs whose name starts with X*
5. *change the name of the blog with Id 5 to 'Bar'*

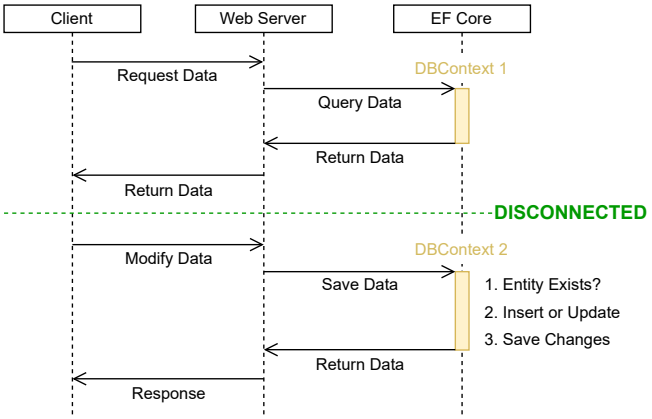
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Overview

Entities often queried with one context instance and saved with another:



New or Existing?

If entity has an **auto-generated key** (IDENTITY, GUID, Sequence):

- Key hasn't been set? Entity must be new ...
- Key has been set? Entity must exist ...

Manually:

```
public static bool IsItNew(Blog blog)
    => blog.BlogId == 0;
```

Built-in:

```
public static bool IsItNew(DbContext context, object entity)
    => !context.Entry(entity).IsKeySet;
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

New or Existing?

If entity has **no auto-generated key**, there are two approaches:

1. Query for the entity using `Find`:

```
public static bool IsItNew(BloggingContext context, Blog blog)
    => context.Blogs.Find(blog.BlogId) == null);
```

2. Pass a flag from the client

```
// Client sends (JSON)
{
  "blogId": 123,
  "title": "Cascading Style Sheets",
  "isNew": false,
  "posts": []
}
```

Auto-generated key

If entity has an **auto-generated key**,
`Update` can be used for insert *and* update:

```
public static void InsertOrUpdate(DbContext context, object entity)
{
    //       👇 if no key value is set, entity is marked for Insert
    context.Update(entity);
    context.SaveChanges();
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

No auto-generated key

The application must choose between insert or update:

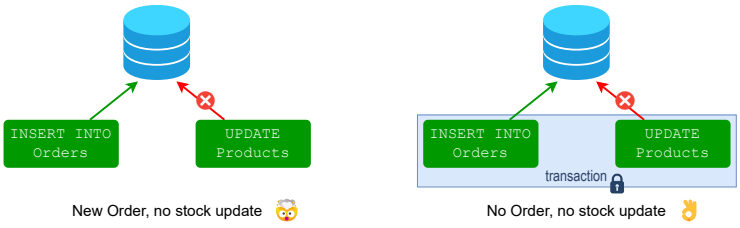
```
public static void InsertOrUpdate(BloggingContext context, Blog blog) {
    var existingBlog = context.Blogs.Find(blog.BlogId);
    if (existingBlog == null) {
        context.Add(blog);                                          // +
    } else {
        context.Entry(existingBlog).CurrentValues.SetValues(blog);  // 
    }
    context.SaveChanges();
}
```

? What is the advantage of using `.SetValues()` ?

Transactions

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Why Transactions?



- At least 2 database modifications applied atomically
  - **committed** = all operations are successfully applied to the db
  - **rolled back** = no operations are applied to the db

Using Transactions

- Implicit
- Explicit
- Ambient
- Fine-grained

- Implicit
- Explicit
- Ambient
- Fine-grained

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Implicit

- Implicit
- Explicit
- Ambient
- Fine-grained

- Every `SaveChanges()` runs automatically inside a transaction
- When error, everything is rolled back automagically

```
context.Users.Add(user);
context.Orders.Add(order);
context.SaveChanges();      // 👈 Implicit transaction
```

For most applications, default behavior is sufficient!

Explicit

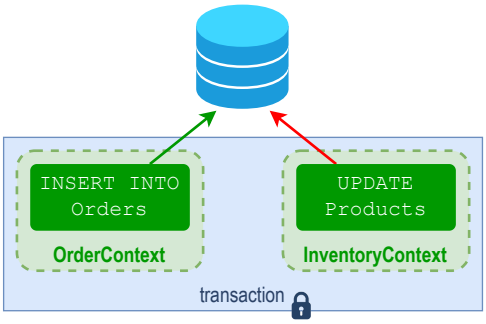
- Implicit
- Explicit
- Ambient
- Fine-grained

When having multiple `SaveChanges()` or conditional commits

```
using var transaction = context.Database.BeginTransaction();

try {
  context.Accounts.Add(account);
  context.SaveChanges();
  context.Payments.Add(payment);
  context.SaveChanges();

  if (/* ... */)
    transaction.Commit()
  else
    transaction.Rollback()

} catch (Exception) {
  // -- Rollback automagically ---
  throw;
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Cross-context transaction



Ambient

- Implicit
- Explicit
- Ambient
- Fine-grained

```
using var scope = new TransactionScope(
    TransactionScopeOption.Required,
    new TransactionOptions { IsolationLevel = IsolationLevel.ReadCommitted },
    TransactionScopeAsyncFlowOption.Enabled);

try {
    using var context1 = new AppDbContext();
    using var context2 = new AppDbContext();

    // Add something on context1 + SaveChanges()
    // Add something on context2 + SaveChanges()

    if (/* ... */) scope.Complete(); // marks transaction for commit
    }
} catch {
    throw;
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Fine-grained control

- Implicit
- Explicit
- Ambient
- **Fine-grained**

```
using (var transaction = new CommittableTransaction(new TransactionOptions {
  IsolationLevel = IsolationLevel.ReadCommitted
})) {
  try {
    // Open context1 + enlist transaction + make changes + SaveChanges()
    // Open context2 + enlist transaction + make changes + SaveChanges()

    transaction.Commit();
  } catch (Exception) {
    // Handle failure, transaction will auto-rollback on dispose
  }
}
```

- Enlist transaction:

```
orderContext.Database.EnlistTransaction(transaction);
```

Key Takeaways

- `SaveChanges` flexible & tracks changes automatically
- Key Concepts:
  - Entity States: `Detached`, `Added`, `Unchanged`, `Modified`, `Deleted`
  - `SaveChanges()` generates SQL based on tracked entity states
- Use `ExecuteUpdate` and `ExecuteDelete` for bulk operations
- When deleting parent or unlinking child, default behavior:
  - required relationship: `CascadeDelete`
  - optional relationship: `ClientSetNull`
- Overwrite default behavior with `DeleteBehavior` enum
- EF Core uses optimistic concurrency by default,
  Changes are detected by using concurrency tokens:
  - database-managed: `.IsRowVersion()`
  - application-managed: `.IsConcurrencyToken()`

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

- Use `DbUpdateConcurrencyException` to resolve conflicts:
  3 value groups: Original Values, Database Values and Current Values:
  - Original = initially retrieved, Current = persist in database
- 3 resolve strategies: db wins, merge, client wins
- `Update()` can be used for insert or update:
  - handy in disconnected scenario's
  - if entities have auto-generated key
- `SetValues` marks only differing properties as `modified`
- Transactions
  - Implicit - `SaveChanges()`
  - Explicit - `Database.BeginTransaction()`
  - Ambient - `TransactionScope`
  - Fine-grained - `EnlistTransaction`

SQL Queries

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Overview

Sometimes written SQL queries are needed in EF Core

- Complex query not possible in LINQ:
  - Window Function, Common Table Expression, ...
- Custom query to improve performance
- Use an existing Stored Procedure or other entity in db

Methods

- FromSql
- SqlQuery
- ExecuteSql

- `FromSql` - returns a mapped entity
- `SqlQuery` - returns scalar or custom type
- `ExecuteSql` - doesn't return data

Each method has a variant ending with 'Raw', for example `FromSqlRaw`

- used for so-called dynamic queries
- unsafe: developer is responsible for preventing SQL injection attacks ⚠️

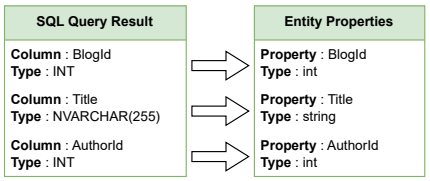
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Basic

- FromSql
- SqlQuery
- ExecuteSql

```
var blogs = context.Blogs
    .FromSql($"SELECT * FROM Blogs")    // 👈 $ is mandatory
    .ToList();
```

- Ensure an exact match: column names & data types



- Only allowed directly on a `DbSet`
- Results are tracked by default
- (Before EF Core 7.0: `FromSqlInterpolated`)

Parameterized query

- FromSql
- SqlQuery
- ExecuteSql

```
var title = "C#";

var blog = context.Blogs
    .FromSql($"SELECT * FROM Blogs WHERE Title = {title}")
    .FirstOrDefault();
```

- Syntax looks like regular C# string interpolation, but isn't!
- Under the hood the supplied value is wrapped in a `DbParameter`
- Makes `FromSql` safe from SQL injection attacks

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

DbParameter

- FromSql
- SqlQuery
- ExecuteSql

Construct a `DbParameter` manually for more control:

```
// var title = "C#";

var title = new SqlParameter
{
  ParameterName = "title",
  SqlDbType = SqlDbType.NVarChar,
  Size = 100,
  Value = "C#"
};

var blog = context.Blogs
  .FromSql($"SELECT * FROM Blogs WHERE Title = {title}")
  .FirstOrDefault();
```

Call Stored Procedure

- FromSql
- SqlQuery
- ExecuteSql

```
var author = "Bart";

var books = context.Blogs
    .FromSql($"EXECUTE dbo.GetMostPopularBlogs @filteredByAuthor={author}")
    .ToList();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Dynamic SQL

- FromSql
- SqlQuery
- ExecuteSql

```
var columnName = "Author";
var columnValue = "johndoe";

var blogs = context.Blogs
    .FromSql($"SELECT * FROM Blogs WHERE {columnName} = {columnValue}")
    .ToList();
```

? This code doesn't run, why?

Dynamic SQL

- FromSql
- SqlQuery
- ExecuteSql

```
var columnName = "Author";
var columnValue = "johndoe";

var blogs = context.Blogs
    .FromSqlRaw($"SELECT * FROM Blogs WHERE {columnName} = {columnValue}")
    .ToList();
```

This code will run!

- `.FromSqlRaw` uses string interpolation for dynamic SQL construction
- Useful when table names or column names need to be parameterized

? Which risks do you see with this code?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Dynamic SQL and SqlParameter

- FromSql
- SqlQuery
- ExecuteSql

Use `SqlParameter` to safely pass parameters

```
var columnName = "Author";                                          // 👈 Safe & validated ⚠️
var columnValue = new SqlParameter("columnValue", "johndoe");  // 👈

var blogs = context.Blogs
    .FromSqlRaw($"SELECT * FROM Blogs WHERE [{columnName}] = @columnValue", columnValue)
    .ToList();
```

❗ Be extremely careful when using `FromSqlRaw`

- Ensure values are from trusted sources or thoroughly sanitized!

Composing with LINQ

- FromSql
- SqlQuery
- ExecuteSql

Compose on top of the initial SQL query using LINQ operators

```
var search = "icecream";

var blogs = context.Blogs
    .FromSql($"SELECT * FROM SearchBlogs({searchTerm})")
    .Include(b => b.Posts)  // 👈
    .ToList();
```

❓ Do you happen to know what db entity is used here to compose over?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Return non-entity types

- FromSql
- SqlQuery
- ExecuteSql

- `SqlQuery` lives on `Database`
- Scalar Type:

```
var blogIds = context.Database
    .SqlQuery<int>(
        @$"SELECT
              BlogId
           FROM Blogs")
    .ToList();
```

- Custom Type:

```
var cutoffDate = new DateOnly(2022, 1, 1);

var summaries = context.Database
    .SqlQuery<PostSummary>(
        @$"SELECT
              b.Name   AS BlogName,
              p.Title  AS PostTitle,
              p.PublishedOn
           FROM Posts AS p
           INNER JOIN Blogs AS b
           ON p.BlogId = b.Id
           WHERE p.PublishedOn >= {cutoffDate}")
    .ToList();
```

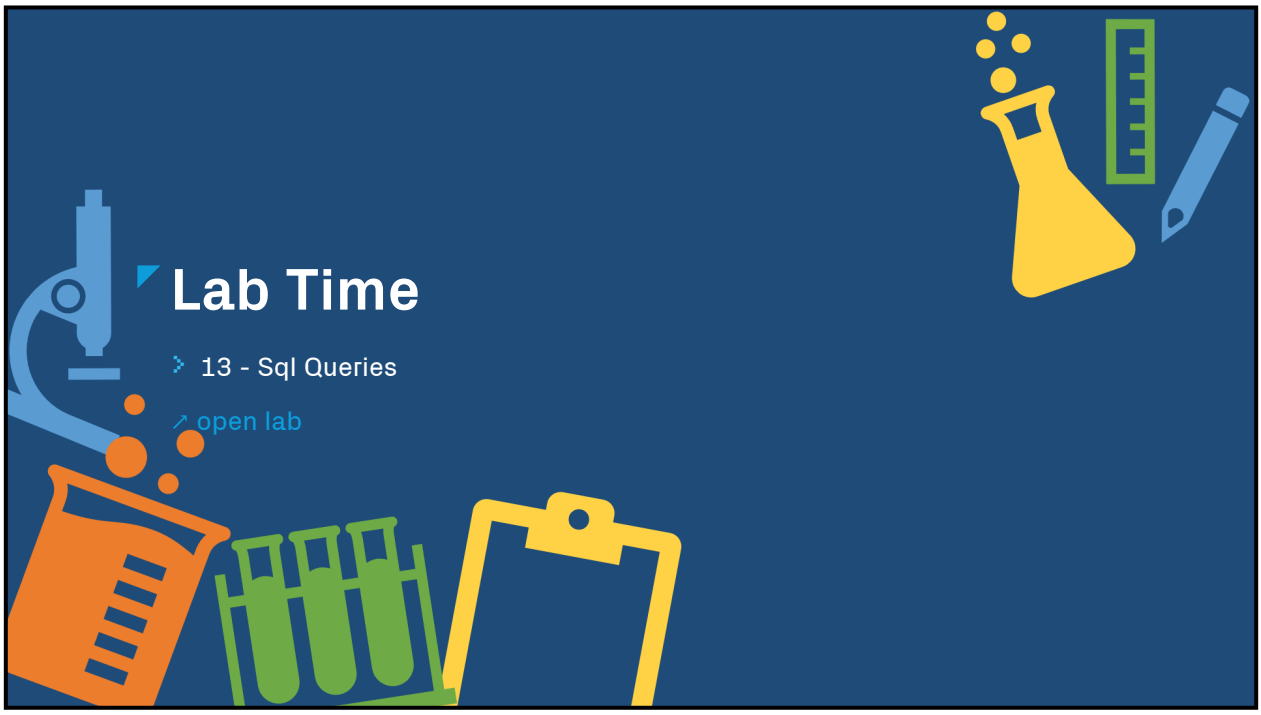
Execute SQL only

- FromSql
- SqlQuery
- ExecuteSql

- `ExecuteSql` does not return any data
- Lives on database

```
context.Database.ExecuteSql(
  @$"CREATE VIEW PostAndBlogSummariesView
     AS
     SELECT b.Name AS BlogName, p.Title AS PostTitle, p.PublishedOn
     FROM Posts AS p
     INNER JOIN Blogs AS b ON p.BlogId = b.Id"
);
```

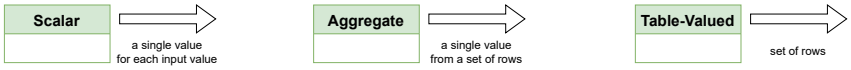
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Database Functions

- Equivalent to C# methods, invoked with parameters to compute results



- Can be user defined
  - `LEN()`
  - `LOWER()`
  - `UPPER()`
  - `FormatZipCode()`
- Only built-in
  - `SUM()`
  - `COUNT()`
  - `MAX()`
  - `AVG()`
- Can be user defined
  - `WeeklyTotalsForCity()`
  - `UnitAverageSalary()`
- Kind of `VIEW` with parameters

Database Function Mappings

- built-in
- EF.Functions
- user-defined

Three ways to map C# functions to database functions

1. Built-in
2. `EF.Functions`
3. User-defined

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Built-in function mapping

- built-in
- EF.Functions
- user-defined

EF Core providers
automatically translate common C# methods to equivalent db functions

`string.ToLower()`	→ `LOWER(column)`
`string.Length()`	→ `LEN(column)`
`numbers.Average()`	→ `AVG(column)`

EF.Functions mapping

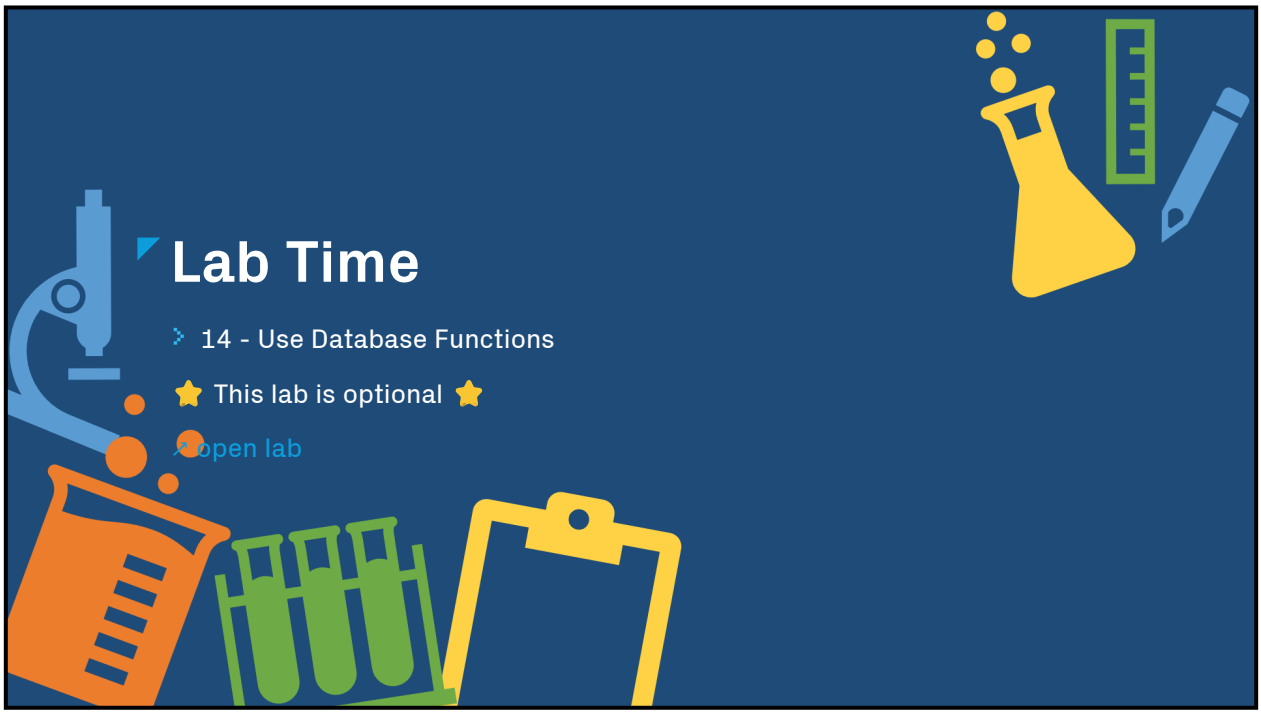
- built-in
- EF.Functions
- user-defined

Special static methods
for database functions that don't have direct C# equivalents

`EF.Functions.Like(name, "%test%")`	→ `LIKE '%test%'`
`EF.Functions.DateDiffDay(start, end)`	→ `DATEDIFF(day, start, end)`

> ! A method that works for one provider may not work for another!

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



UDF and TVF

- built-in
- EF.Functions
- user-defined

Many database systems support the creation of custom functions:

1. User-Defined Function

```
CREATE FUNCTION dbo.AddOne (@value INT)
RETURNS INT
AS
BEGIN
    RETURN @value + 1
END
```

2. Table-Valued Function

```
CREATE FUNCTION dbo.GetActiveUsers()
RETURNS TABLE
AS
RETURN (
  SELECT * FROM Users
  WHERE IsActive = 1
)
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

User-defined function mapping

> Why can't you use **built-in** or **EF.Functions** mappings for calling your own db functions from LINQ?

- Link a SQL function to a C# method in your model setup
- When EF Core converts a LINQ query to SQL, it will call the SQL function (not the C# method)
- Can be used for UDFs and TVFs

Steps for UDF

1 Create DB function

```
CREATE FUNCTION dbo.AddOne (@value INT)
RETURNS INT AS
BEGIN
    RETURN @value + 1
END
```

2 Add C# method in `DbContext`

```
public int AddOne(int value)
  => throw new NotImplementedException();
```

- Only signature matters!
- Acts as proxy

3 Register function mapping

Fluent API

```
modelBuilder
  .HasDbFunction(() => AddOne(default));
```

4 Use mapped method in LINQ

```
var results = context.Entities
    .Where(e => MyContext.AddOne(e.SomeIntColumn))
    .ToList();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Steps for TVF

- built-in
- EF.Functions
- user-defined

1 Create DB function

```
CREATE FUNCTION dbo.GetActiveUsers()
RETURNS TABLE
AS
RETURN
(
  SELECT * FROM Users
  WHERE IsActive = 1
)
```

2 Add C# method in `DbContext`

```
public IQueryable<User> GetActiveUsers()
  => FromExpression(() => GetActiveUsers());
```

- `FromExpression()` never executed
- translated to expression tree

3 Register function mapping

Fluent API

```
modelBuilder
  .HasDbFunction(() => GetActiveUsers());
```

4 Use mapped method in LINQ

```
var activeUsers = context.GetActiveUsers()
                         .ToList();
```

Global Query Filters

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Common scenarios

Soft delete

- In stead of deleting records, a flag is set (like `IsDeleted`)
- Keep a history of deleted records for compliance purposes

Multi-tenancy

- SaaS solution -> many customers (tenants) may use same app & db
- Their data should be kept private and separate

> ? Do you want to add `Where(...)` to every LINQ query yourself?

🔧 Global Query Filters

- Use `HasQueryFilter` API
- Automatically excludes all "deleted" posts from queries

Fluent API

```
//public property
modelBuilder.Entity<Post>()
  .HasQueryFilter(p => !p.IsDeleted);
```

- Automatically filters blogs by tenant ID

Fluent API

```
//private field
modelBuilder.Entity<Blog>()
  .HasQueryFilter(b => EF.Property<string>(b, "_tenantId") == _tenantId);
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Use navigations

Navigation in a query filter will cause query filters to be applied recursively

```
modelBuilder.Entity<Blog>().HasMany(b => b.Posts).WithOne(p => p.Blog);
modelBuilder.Entity<Blog>().HasQueryFilter(b => b.Posts.Count > 0);
modelBuilder.Entity<Post>().HasQueryFilter(p => p.Title.Contains("fish"));
```

```
var filteredBlogs = db.Blogs
  .ToList();
```

```
SELECT
    b.BlogId, b.Name, b.Url
FROM Blogs AS b
WHERE
(
    SELECT COUNT(*)
    FROM Posts AS p
    WHERE p.Title LIKE '%fish%'
    AND b.BlogId = p.BlogId
) > 0
```

Good to know

- In inheritance, filters can only be set on the root entity type
- Multiple query filters on 1 entity not possible, only last one will be applied
  Solution: Define a single filter with multiple conditions using `&&`
- Disable filter for individual LINQ queries:

```
blogs = db.Blogs
    .Include(b => b.Posts)
    .IgnoreQueryFilters()   // 👈
    .ToList();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

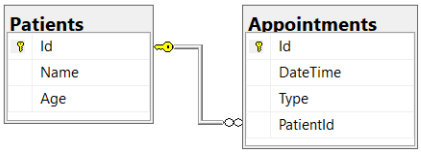
Complex Query Operators

Intro

- LINQ has many complex operators
- Not all operators can be translated to SQL by EF Core
- Translation support depends on db provider
- How a query is written can affect whether it translates (even if the result is the same)

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Simple diagram



GroupBy

- EF Core groups **after** fetching data from the db

```
var appointmentsByPatient = context.Appointments
    .GroupBy(a => a.PatientId);
```

```
SELECT
    a.PatientId, a.Id,
    a.DateTime, a.Type
FROM Appointments AS a
ORDER BY a.PatientId
-- EF Core groups afterwards ...
```

No database structure can represent an `IGrouping` 🤔
in most cases `GroupBy` operators have no translation, except ...

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

GROUP BY

- GROUP BY
- INNER JOIN
- LEFT OUTER JOIN
- CROSS JOIN

`GroupBy` becomes `GROUP BY` when:

- Each group uses an aggregate like `Count`, `Sum`, `Min`, `Max`, `Average`
- Grouping is only by simple (scalar) values
- Result only includes group keys and/or aggregates

```
var appointmentsByPatient = await context.Appointments
  .GroupBy(a => a.PatientId) // 👈 scalar value
  .Select(g => new
  {
      Key = g.Key,           // 👈 group key
      Count = g.Count()      // 👈 aggregate
  });
```

```
SELECT
    a.PatientId  AS Key,
    COUNT(*)     AS Count
FROM Appointments AS a
GROUP BY a.PatientId
```

HAVING

- GROUP BY
- INNER JOIN
- LEFT OUTER JOIN
- CROSS JOIN

`Where` with aggregate translates to `HAVING`

```
var appointmentsByPatient = await context.Appointments
  .GroupBy(a => a.PatientId)
  .Where(g => g.Count() > 0)   // 👈
  .Select(g => new
  {
      Key = g.Key,
      Count = g.Count()
  });
```

```
SELECT
    a.PatientId  AS Key,
    COUNT(*)     AS Count
FROM Appointments AS a
GROUP BY a.PatientId
HAVING COUNT(*) > 0   // 👈
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

INNER JOIN

Always prefer `Include()`, but what if there are:

- no navigation properties
- no relationships configured with Fluent API

`Join()` to the rescue! 🦹

```
var results = context.Patients
    .Join(
        context.Appointments,
        patient => patient.Id,
        appointment => appointment.PatientId,
        (patient, appointment) =>
          new { patient, appointment }
    );
```

```
SELECT p.Id, p.Age, p.Name,
       a.Id, a.DateTime,
       a.PatientId, a.Type
FROM Patients AS p
INNER JOIN Appointments AS a
ON p.Id = a.PatientId
```

LEFT OUTER JOIN

- Very similar to `INNER JOIN`

```
var results = context.Patients
    .LeftJoin(
        context.Appointments,
        patient => patient.Id,
        appointment => appointment.PatientId,
        (patient, appointment) => new { patient, appointment }
    )
    .OrderBy(pa => pa.patient.Name)
    .ThenBy(pa => pa.appointment != null
                  ? pa.appointment.DateTime
                  : string.Empty);
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

CROSS JOIN

- Use `.SelectMany()`
- Flattens child collections into a single merged collection
- Cartesian Product

```
var results = context.Patients
    .SelectMany(
        patient => context.Appointments,
        (p, a) => new { Patient = p, Appointment = a }
    );
```

```
SELECT
    p.Id, p.Age, p.Name,
    a.Id, a.DateTime,
    a.PatientId, a.Type
FROM Patients AS p
CROSS JOIN Appointments AS a
```

Accessing Tracked Entities

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

APIs

APIs for accessing **tracked entities**:

1. `DbContext.Entry(entity)`
2. `ChangeTracker.Entries()`
3. `DbContext.Find(key)`
4. `DbSet<T>.Local`

`EntityEntry` instance

EF Core tracks the following information for each tracked entity:

- **State**
- **Relationships**: Navigation properties to other tracked entities
- **Current values**: In-memory property values
- **Original values**: Values when queried from database
- **Modified properties**: Which properties changed since querying

This info is stored in an `EntityEntry` instance

> ? In which states can a tracked entity be?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

DbContext.Entry

- DbContext.Entry(ent
- ChangeTracker.Entri
- DbContext.Find(key)
- DbSet<T>.Local

- Returns an `EntityEntry<TEntity>` with tracking info for that entity

```
using var context = new BlogsContext();

var blog = context.Blogs.Find(1);
var entityEntry = context.Entry(blog);   // Returns EntityEntry<Blog>

if (entityEntry.State == EntityState.Unchanged)
{
    context.Entry(blog).State = EntityState.Modified;   // Change state
}
```

PropertyEntry

- DbContext.Entry(ent
- ChangeTracker.Entri
- DbContext.Find(key)
- DbSet<T>.Local

Access individual property tracking info via:

```
var propertyEntry = context.Entry(blog).Property(b => b.Name);
// or
var propertyEntry = context.Entry(blog).Property("Name");
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

PropertyEntry

- Original value:
  - Value when entity was queried
  - Not available for disconnected entities (returns current value instead)
- IsModified:
  - `true` → force update property
  - `false` → prevent update property

```
var nameProperty = context.Entry(blog).Property(b => b.Name);
var originalName = nameProperty.OriginalValue;
nameProperty.IsModified = true;  // Force update
```

- DbContext.Entry(ent
- ChangeTracker.Entri
- DbContext.Find(key)
- DbSet<T>.Local

Iterate All Properties

`EntityEntry.Properties` returns all `PropertyEntry` instances for an entity

```
// Update all DateTime properties to now
foreach (var property in context.Entry(blog).Properties)
{
    if (property.Metadata.ClrType == typeof(DateTime))
    {
        property.CurrentValue = DateTime.Now;
    }
}
```

- DbContext.Entry(ent
- ChangeTracker.Entri
- DbContext.Find(key)
- DbSet<T>.Local

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Property Values

- Get property values in different states:

```
var currentValues = context.Entry(blog).CurrentValues;        // In-memory values
var originalValues = context.Entry(blog).OriginalValues;      // Values when queried
var databaseValues = context.Entry(blog).GetDatabaseValues(); // Current DB values
```

- Map between entities and DTOs:

```
// Copy DTO values to entity
context.Entry(blog).CurrentValues.SetValues(blogDto);
```

- Detect and resolve optimistic concurrency conflicts

ChangeTracker.Entries

Returns `EntityEntry` for every tracked entity:

```
var blogs = context.Blogs.Include(e => e.Posts).ToList();

foreach (var entry in context.ChangeTracker.Entries())
{
    Console.WriteLine($"{entry.Metadata.Name} with ID " +
                      $"{entry.Property("Id").CurrentValue}");
}
```

```
Blog with ID 1
Post with ID 1
Post with ID 2
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

`Find` and `FindAsync`

Efficient lookup by primary key:

1. Checks tracked entities first (no DB query)
2. Queries database only if not tracked

```
var blog1 = context.Blogs.Find(1);          // Query DB
Console.WriteLine($"Found: {blog1.Name}");

var blog2 = context.Blogs.Find(1);          // Returns cached instance
Debug.Assert(blog1 == blog2);               // Same instance
```

Performance benefit: Second call skips database query entirely

`DbSet<T>.Local`

- Get tracked entities of a specific type without `EntityEntry` wrapper:

```
context.Blogs.Include(e => e.Posts).Load();

// ChangeTracker.Entries
foreach (var entry in context.ChangeTracker.Entries<Blog>())
    Console.WriteLine(entry.Entity.Name);

// DbSet<T>.Local
foreach (var blog in context.Blogs.Local)
    Console.WriteLine(blog.Name);
```

- Type-safe: only `Blog` or `Post` entities
- Ideal for data binding scenarios

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

- 3 'backdoors' in EF Core:
  - `FromSql` for mapped entities
  - `SqlQuery` for scalars or custom types
  - `ExecuteSql` for commands that don't return data
- Always parameterize SQL queries to prevent SQL injection
- C# methods like `ToLower()`, `Length()`, `Average()` are translated to SQL automatically
- Use `EF.Functions` for special DB functions: `Like`, `DateDiffDay`, ...
- Map custom DB functions:
  - create C# method
  - register it with `HasDbFunction`
  - calls to this method in LINQ are translated to corresponding SQL function

Key Takeaways

- Global query filters
  - Define `Where`-conditions for *soft-delete*, *multi-tenancy*, once ...
  - ... and they are applied automatically to all queries for that entity
  - Define with `HasQueryFilter` in `OnModelCreating`
- `GroupBy` groups elements by a key
- `Join` combines two collections based on matching keys (inner join)
- `SelectMany` flattens collections of collections into a single sequence (cross join)

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

- Accessing Tracked Entities:
  - `context.Entry(entity)` - Get tracking info for one entity
  - `context.ChangeTracker.Entries()` - Get all tracked entities
  - `context.Find(id)` - Efficient lookup (cache first, then DB)
  - `context.DbSet.Local` - Get tracked entities by type

Migrations

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Why Migrations?

- Keep db schema in sync with data model
- Preserve existing data during updates
- Improved collaboration

Steps

- ✨ Model has changed
- 🛠️ Create a Migration
- 🔍 EF Core compares the current model with a previous snapshot
- 📝 EF Core generates Migration files describing the changes
- 📁 Add Migration files to source control
- 🚀 Apply the Migration
- 📋 EF Core uses a history table to track applied migrations

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

CLI or PMC

Choose **CLI** or **Package Manager Console**

1. **CLI** (`dotnet ef`)
   -
2. **Package Manager Console**
   - only
   - provides Intellisense

- Install CLI:

```
dotnet tool install --global dotnet-ef
```

Install package

- In the project containing `DbContext`:

.NET Core CLI

```
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Visual Studio

```
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.EntityFrameworkCore.Design
```

Visual Studio: Only install `Design` if `DbContext` is not in the startup project

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Allow Migrations

- Update `DbContext`:

```
public AppDbContext(DbContextOptions<AppDbContext> contextOptions):
    base(contextOptions)
{}
public AppDbContext()    // 👈 Add parameterless constructor for migrations
{
}
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
  if (!optionsBuilder.IsConfigured)
  {
    optionsBuilder.UseSqlServer(/* conn string */);
                                    // 👈 ONLY used during migrations
  }
}
```

Create first Migration

- With name 'InitialCreate'

```
dotnet ef migrations add InitialCreate
```

or

```
Add-Migration InitialCreate
```

- EF Core creates a directory called `Migrations` and generates
  - Migration class
  - Snapshot

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Apply Migration

```
dotnet ef database update
```

or

```
Update-Database
```

- If database does not exist, EF Core creates it
- A **migration history table** is added to track applied migrations

Evolving model

- Create a new migration when model and database are out of sync

```
dotnet ef migrations add AddBlogCreatedTimestamp
```

or

```
Add-Migration AddBlogCreatedTimestamp
```

- EF Core compares the updated model to the previous snapshot
- It detects changes and creates a migration for them

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

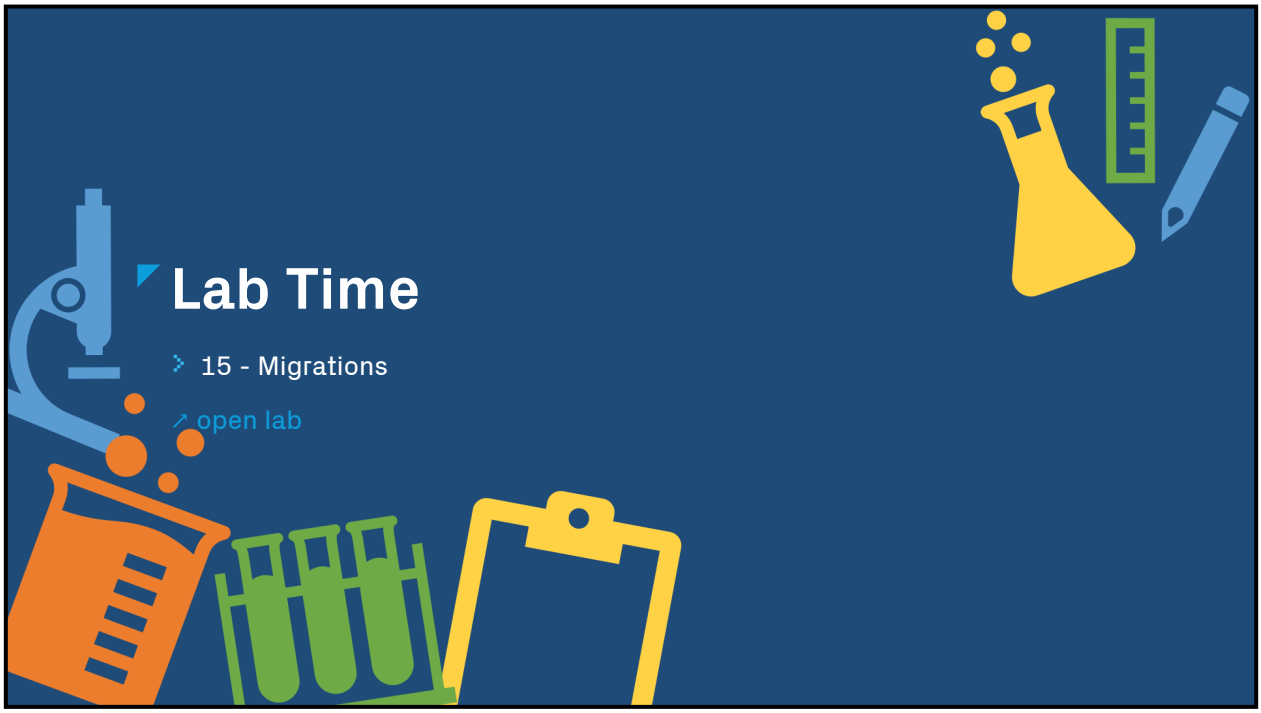
Apply Migration

```
dotnet ef database update
```

or

```
Update-Database
```

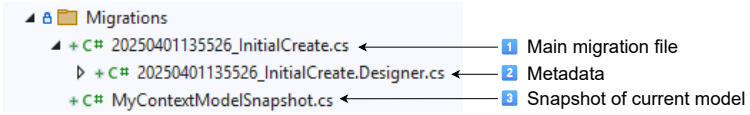
- If the database exists, EF Core checks the migrations history table
  - Only new migrations are applied



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Manage Migrations

Add a migration



1. **DateTimeStamp_MigrationName.cs**
   - Operations necessary to apply the migration (`Up`) and to revert it (`Down`)
2. **DateTimeStamp_MigrationName.Designer.cs**
   - Contains information used by EF Core
3. **ContextNameModelSnapshot.cs**
   - Used to detect model changes when generating next migration

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Namespaces

- Move Migration files to a different folder and manually update their namespace manually if preferred, or
- Specify the directory at generation time:

```
dotnet ef migrations add InitialCreate --output-dir Your/Directory
```

or

```
Add-Migration InitialCreate -OutputDir Your\Directory
```

Customize migration code

- EF Core usually generates correct migrations
- **Always review the code** 🔍
- Customizing is required is when renaming a property
- Replace the generated migration:

```
migrationBuilder.RenameColumn(
    name: "Name",
    table: "Customers",
    newName: "FullName");
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Raw SQL

- Merge 2 columns in a database schema update

```
migrationBuilder.AddColumn<string>(
    name: "FullName", table: "Customer", nullable: true);

migrationBuilder.Sql(@"
  UPDATE Customer
  SET FullName = FirstName + ' ' + LastName;
");

migrationBuilder.DropColumn(name: "FirstName", table: "Customer");
migrationBuilder.DropColumn(name: "LastName", table: "Customer");
```

Raw SQL

- Manage database objects not tracked by EF Core:
  stored procedures, functions, views ...
- Add migration with no model changes, fill it with raw SQL:

```
migrationBuilder.Sql(@"
    CREATE PROCEDURE getFullName
        @LastName nvarchar(50),
        @FirstName nvarchar(50)
    AS
    RETURN @LastName + @FirstName
");
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Remove last Migration

```
dotnet ef migrations remove
```

or

```
Remove-Migration
```

Afterward make the additional model changes and add it again

> Avoid removing migrations already applied to production databases

List migrations

```
dotnet ef migrations list
```

or

```
Get-Migration
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Reset all migrations

- Delete `Migrations` folder
- Drop your database
- Create a new initial migration

> ❗ Custom migration code will be lost as well

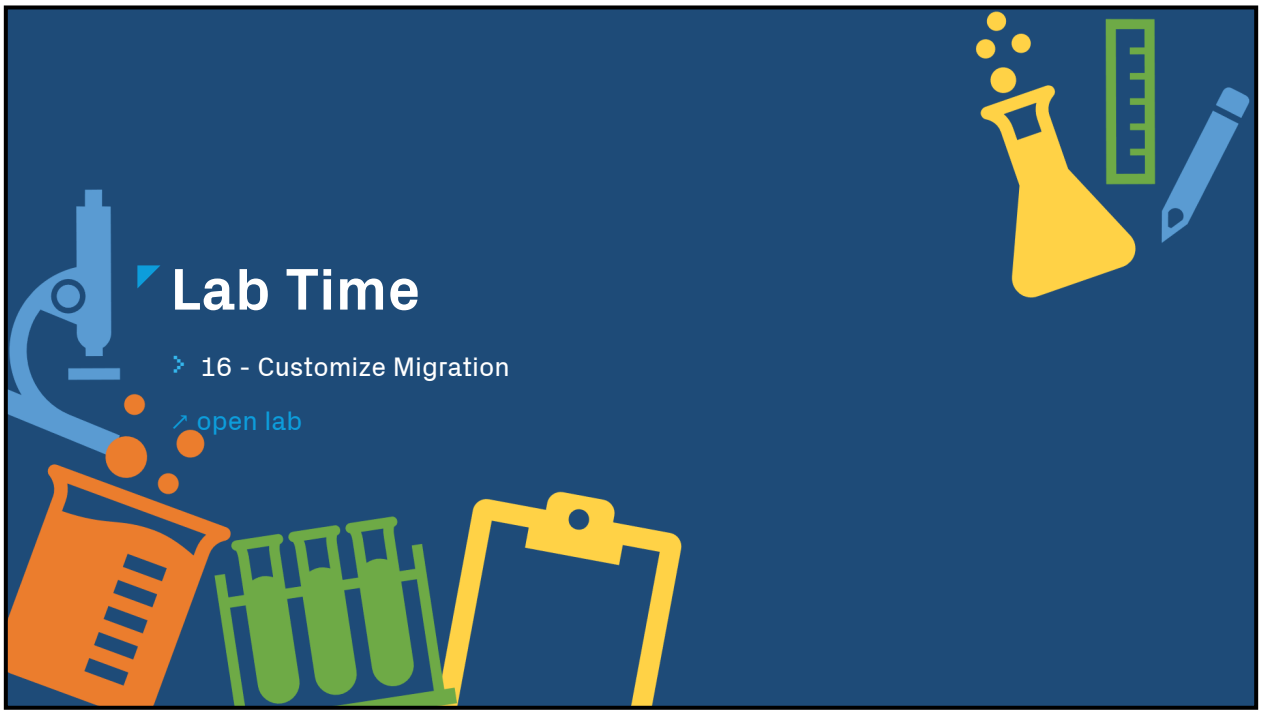
Squashing

Consolidate multiple migrations into a single initial migration:

1. Back up the database
2. Delete all rows from the migrations history table
3. Delete `Migrations` folder
4. Create a new migration
5. Insert a single row into the migrations history:

```
INSERT INTO [__EFMigrationsHistory] ([MIGRATIONID], [PRODUCTVERSION])
VALUES (N'<full_migration_timestamp_and_name>', N'<EF_version>');
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Data Seeding

- Migration
- Model
- Initialization

Populate database with an initial set of data

EF Core supports three data seeding methods:

- Customizing migrations
- Model seed data
- Custom initialization logic

? Which ones have we used thusfar?

Manual migration customization

- Migration
- Model
- Initialization

- Handy for ensuring seed data is added as part of a specific migration:

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // ... generate table Blogs ...

    migrationBuilder.InsertData(   // 👈
        table: "Blogs",
        columns: new[] { "Url" },
        values: new object[] { "http://generated.com" }
    );
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Model seed data

- Migration
- Model
- Initialization

- For small, static read-only datasets
- EF Core compares the model snapshot to the current model and generates `InsertData()`, `UpdateData()` and `DeleteData()` operations

> ❗ Only changes in the model are tracked
> Manual database edits may be lost or cause errors

Provide Keys explicitly

- Migration
- Model
- Initialization

- Define initial data in `OnModelCreating`, using `HasData`
- Specify Primary Key 🔑 even if it's configured as auto-generated

```
modelBuilder.Entity<Blog>()
  .HasData(
     new Blog { BlogId = 1, Url = "http://sample.com" }   // 🔑 BlogId
  );
```

- Similarly, specify the Foreign Key explicitly when seeding related entities

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Shadow Properties

- Migration
- **Model**
- Initialization

- If an entity type has any shadow property, use an anonymous class to provide the values:

```
modelBuilder.Entity<Post>()
  .HasData(
    // new Post { BlogId = 1, PostId = 2, Title = "Second post", Content = "Test 2" }
    new { BlogId = 1, PostId = 2, Title = "Second post", Content = "Test 2" }
  );
```

? What is a shadow property in EF Core?

Custom Initialization logic

- Migration
- Model
- **Initialization**

Maximum flexibility!

- Data depends on existing database state
- Large data sets
- Keys must be generated by the database
- Custom transformations needed (like password hashing)
- When external API calls are required (like creating Identity Roles)

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

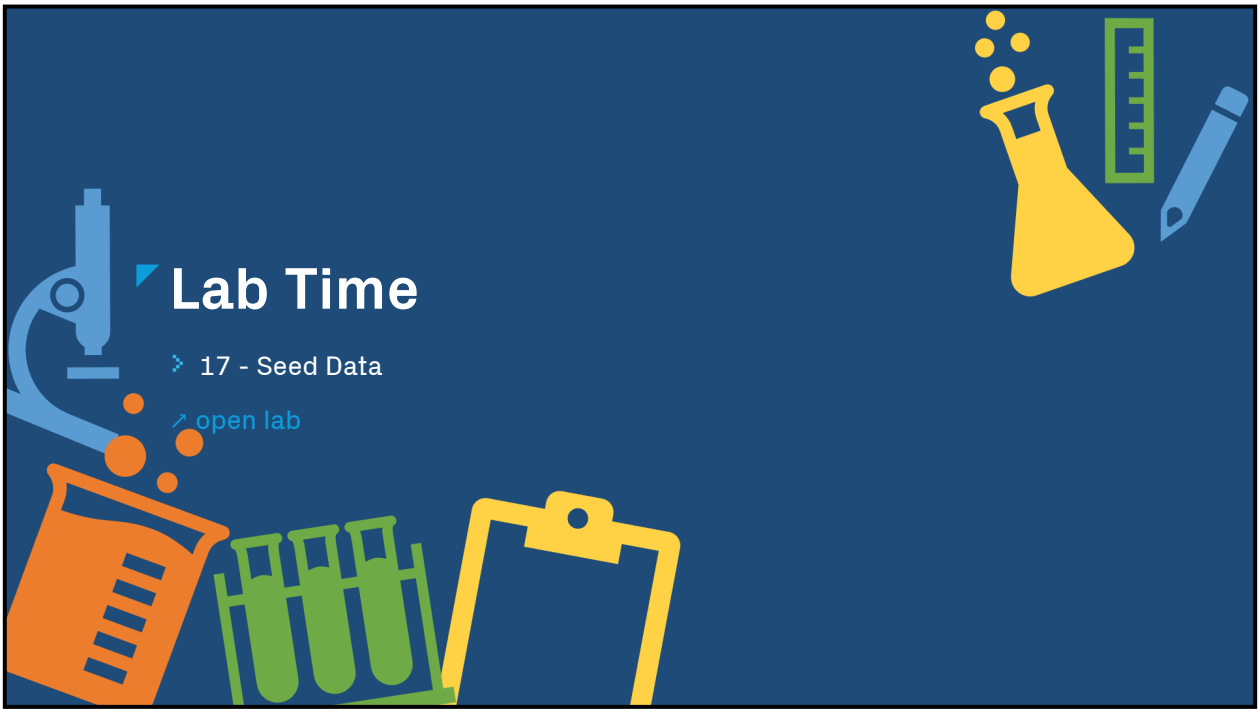
Custom initialization logic

- Migration
- Model
- Initialization

Seed data using `DbContext.SaveChanges()` before main app logic runs in `UseSeeding()`/`UseAsyncSeeding()`

```
optionsBuilder.UseSqlServer(configuration.GetConnectionString("DefaultConn"))
.UseSeeding( (context, _ ) =>
{
  var testBlog = context
      .Blogs
      .FirstOrDefault(b => b.Url == "http://test.com");

  if (testBlog == null) {
      context.Blogs.Add(new Blog { Url = "http://test.com" });
  }

  context.SaveChanges();
});
```

Seeding code should not be part of the normal app execution



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Apply Migrations

Apply migrations

- At runtime
- Command-line
- SQL scripts

Various strategies to deploy and apply migrations

Local development

- Apply the migration at runtime *or*
- Via the command-line tools

Production

- Let SQL scripts be generated to run on the database

? Where would you test your migrations?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Apply at runtime

- At runtime
- Command-line
- SQL scripts

- For local development and testing of migrations

```
// Console App
using var context = new AppDbContext(options);
context.Database.Migrate();   // 👈 Apply pending migrations
```

```
// ASP.NET Core App
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();   // 👈 Apply pending migrations
}
```

Apply at runtime

- At runtime
- Command-line
- SQL scripts

❗ Not recommended for production:

- Multiple app instances may try to migrate at the same time => errors
- The app needs high database permissions
- No chance to review or modify the SQL before it runs
- Harder to roll back if something goes wrong

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Command-line tools

- At runtime
- Command-line
- SQL scripts

- For local development and testing of migrations

> ! Again, not ideal for production:

- Same reasons as when applying migrations at runtime
- Requires .NET SDK & source code on the server

Command-line tools

- At runtime
- Command-line
- SQL scripts

```
dotnet ef database update               # update db to the latest migration
dotnet ef database update AddNewTables  # Update db to given migration
```

or

```
Update-Database
Update-Database AddNewTables
```

- Can be used to roll back to an earlier migration as well

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

SQL scripts

Generate SQL scripts from your migrations
Apply them manually or as part of a deployment pipeline

- Recommended for production
- Can be reviewed, tested and archived
- Can be provided to a DBA
- Can be customized for production needs

Generate SQL Script


```
dotnet ef migrations script                              # basic usage
dotnet ef migrations script AddNewTables AddAuditTable   # from / to
dotnet ef migrations script --idempotent                 # idempotent script
```

or

```
Script-Migration
Script-Migration -From AddNewTables -To AddAuditTable
Script-Migration -Idempotent
```

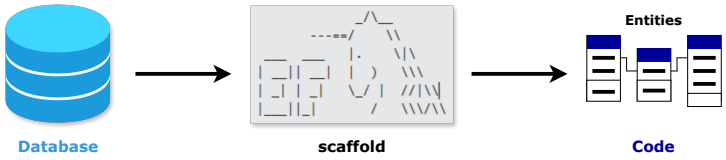
- To generate a **rollback script**, set `-From` to a newer migration than `-To`

? What is the meaning of the `-Idempotent` flag

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

DB-First

DB-First



- AKA Reverse engineering
- Scaffold entity types + `DbContext` from existing database

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Scaffold

- NuGet Packages:
  - Microsoft.EntityFrameworkCore.SqlServer
  - Microsoft.EntityFrameworkCore.Tools
- View all options:

```
get-help Scaffold-DbContext –full
```

- Example:

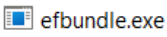
```
Scaffold-DbContext
"Server=.\sqlexpress;Database=Blogging;Trusted_Connection=True;"
-Provider "Microsoft.EntityFrameworkCore.SqlServer"
```

Bundles

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Migration bundle

Single-file executable that can apply migrations to a database



- One file - simple & portable deployment
- No source code required - safe & easy distribution
- Can be generated as part of a CI process
- Can run without EF Core Tools / .NET SDK / .NET Runtime

Advantages

Compared to SQL script and command-line tools:

- No extra tools needed
- Handle transactions and errors more consistently
  Reduce the risk of leaving your database in a broken or undefined state

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Command-line tools

Generate a bundle

```
dotnet ef migrations bundle          # the command
dotnet ef migrations bundle --help   # detailed info

# Generates a self-contained bundle for Linux:
dotnet ef migrations bundle --self-contained -r linux-x64
```

or

```
Bundle-Migration
Get-Help Bundle-Migration -Detailed

Bundle-Migration -SelfContained -TargetRuntime linux-x64
```

efbundle

- Generated executable is called `efbundle` by default
- Running it updates the database to the latest migration
- Works the same as `dotnet ef database update` or `Update-Database`, but without source code or EF Core tools

```
C:\>efbundle.exe --help
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Example

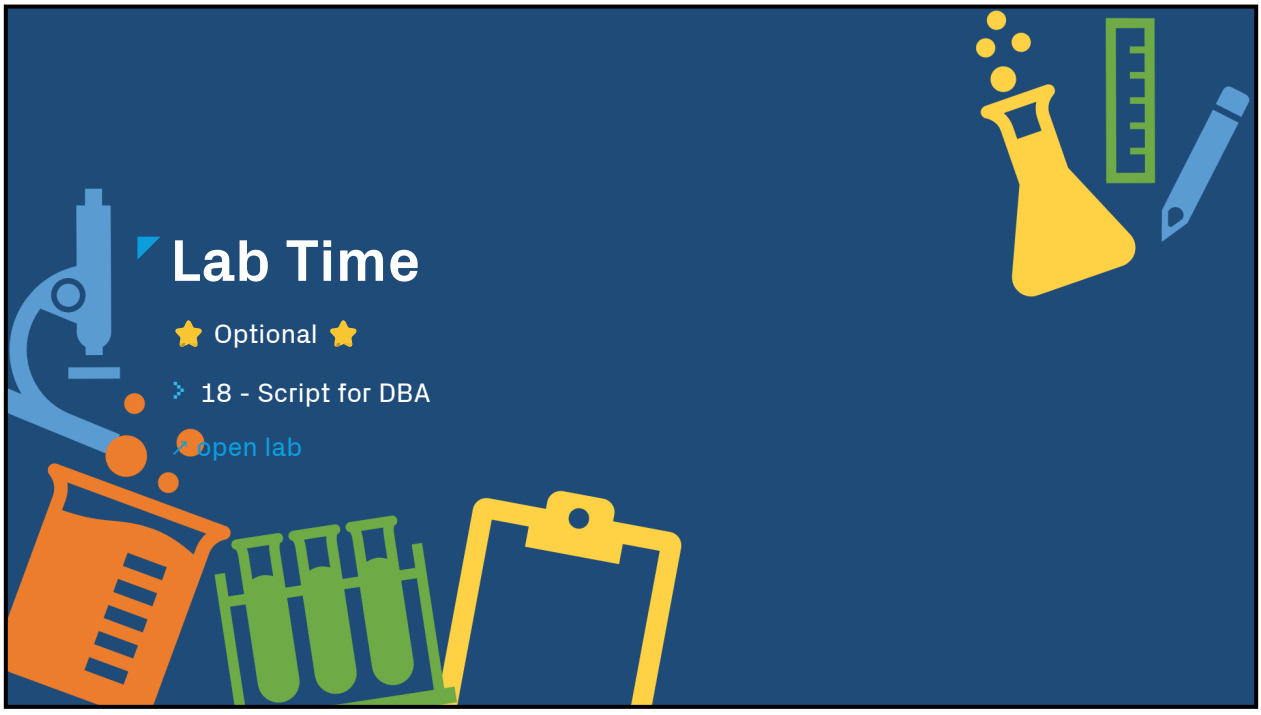
Apply migrations to a local SQL Server instance:

- Option 1: Provide connection string:

```
.\efbundle.exe --connection 'Data Source=(local)\MSSQLSERVER;
Initial Catalog=Blogging;User ID=myUsername;Password=myPassword'
```

- Option 2: Use `appsettings.json` alongside your bundle:

```
.\efbundle.exe
```



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

- **Create & Manage Migrations**
  - `Add-Migration` - create migrations
  - Always review generated code - customize/rename when needed
  - `Remove-Migration` - remove last migration
- **Data Seeding**
  - Use `HasData()` in `OnModelCreating` for model seed data
  - Explicitly specify primary and foreign keys when seeding
- **Apply Migrations**
  - Runtime: `context.Database.Migrate()` (development only)
  - Command-line: `Update-Database`
  - SQL Scripts: `Script-Migration` (**recommended for production**), tip: `--idempotent`
  - Bundles: `Bundle-Migration` (portable deployment)
- **Database-First**
  - Opposite of Code-First
  - `Scaffold-DbContext`

Logging

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Logging

- Simple
- Extensions
- Events

Choose the right tool for your needs:

- **Simple Logging** - Development-time debugging
- **Microsoft.Extensions.Logging** - Production logging (ASP.NET Core default)
- **Events** - React to EF Core operations

Simple Logging

- Simple
- Extensions
- Events

Use `LogTo` in `DbContext.OnConfiguring` to enable simple logging:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.LogTo(Console.WriteLine);
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Microsoft.Extensions.Logging

- Production-ready logging with pluggable providers:
  - Microsoft providers:
    `Microsoft.Extensions.Logging.Console`, `Debug`, `EventLog`
  - Third-party providers:
    `Serilog`, `NLog`, `log4net`
- Default in ASP.NET Core(automatically configured)

Events

React to EF Core operations using .NET events:

- Subscribe to events like `SavedChanges`, `StateChanged`
- Synchronous only (no async I/O)
- Registered per `DbContext` instance

> For application-wide event handling, use **diagnostic listeners** instead.

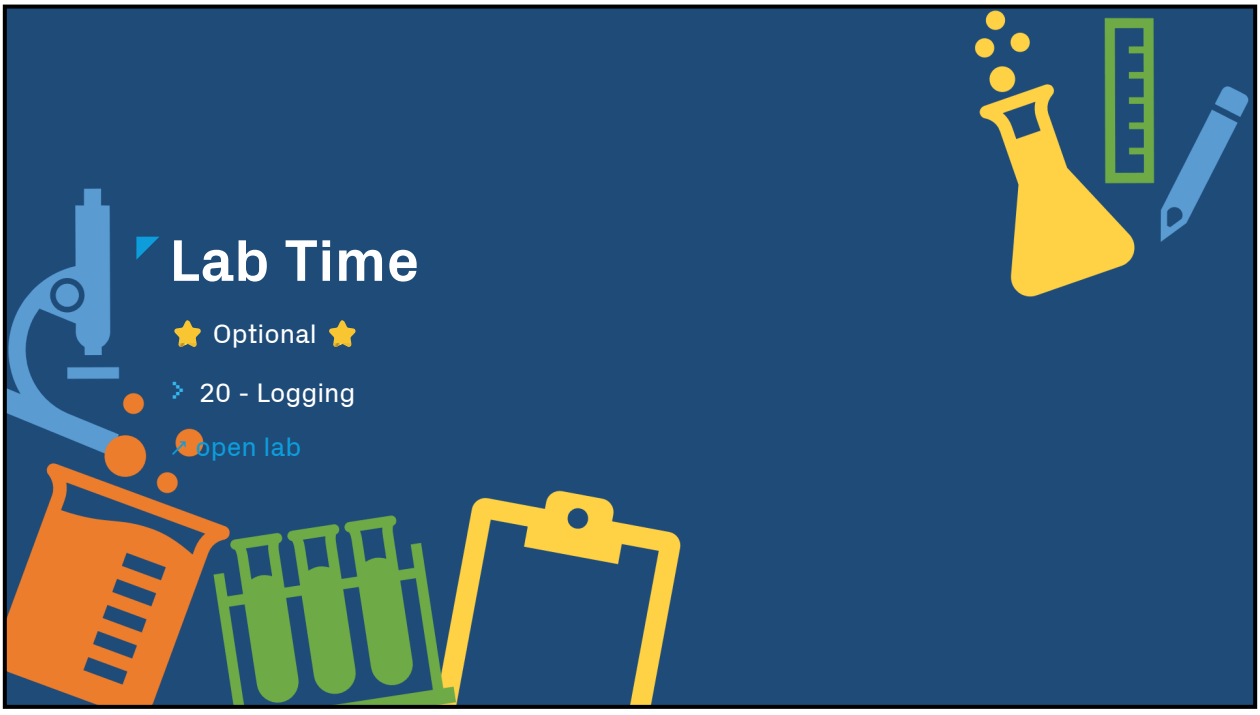
© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Events Example

- Simple
- Extensions
- Events

- React to `SaveChanges`:

```
public class BlogsContext : DbContext
{
 public BlogsContext()
 {
     // Register event handler
     this.SavedChanges += OnSavedChanges;
 }

 private void OnSavedChanges(object? sender, SavedChangesEventArgs e)
 {
     Console.WriteLine($"Saved {e.EntitiesSavedCount} entities");
 }
}
```



© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways:

- Simple Logging:
  - Quick dev debugging, inspect SQL queries locally
- Microsoft.Extensions.Logging:
  - Production apps, structured logs, multiple outputs
- Events:
  - Audit trails, custom SaveChanges logic, monitoring

Testing

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Integration Testing with EF Core

- Test how components work together, including *real database operations*
- What we test:
  - Database queries return correct data
  - Relationships and navigation properties work
  - Transactions and SaveChanges behave correctly
  - Migrations apply successfully

- Unit tests:
  - Mock the database (fast, isolated)
- Integration tests:
  - Use a real database provider (realistic, slower)

Integration Test

- Configure your DbContext
  - 👉 *more in next slides*
- Pass it to your SUT
- Exercise your SUT
- Test the results

```
[Fact]
public void CreateBlog_AddsBlogToDatabase() {
  // Arrange
  using var context = GetContext();
  var sut = new BlogsRepository(context);
  var blog = new Blog { Title = "New Blog" };

  // Act
  sut.CreateBlog(blog);

  //Assert
  var blogs = sut.GetAllBlogs();
  Assert.Single(blogs);
  Assert.Equal("New Blog", blogs.First().Title);
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Three main options

- In-Memory
- SQLite
- Test Containers

- EF Core In-Memory
- SQLite (in-memory)
- Test Containers

EF Core In-Memory

- In-Memory
- SQLite
- Test Containers

✅ Advantages:

- Built into EF Core
- Easy to setup
- Fast execution

❌ Disadvantages:

- Behavior differs significantly from real databases
  - No enforcement of Foreign Keys
  - No transaction support
  - No `FromSqlRaw`

❗ Microsoft strongly discourages using this provider in new projects

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

SQLite (in-memory)

✅ Advantages:

- Easy to setup
- Fast execution
- Real relational database
- Enforces constraints
- Supports transactions

❌ Disadvantages:

- Not the same engine as SQL Server/PostgreSQL/...
- Potential differences in behavior (e.g., string comparisons, date handling)

ℹ Good compromise for integration tests when real db isn't practical

SQLite (in-memory)


```
public class BlogsRepositoryInMemoryTests {
  public BlogsRepositoryInMemoryTests(){
    _connection = new SqliteConnection("Filename=:memory:");
    _connection.Open();
    _contextOptions = new DbContextOptionsBuilder<BloggingContext>()
        .UseSqlite(_connection)
        .Options;
  }
  private BlogsContext GetContext() => new BlogsContext(options);
  public void Dispose() => _connection.Dispose();
  // ... tests here
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Test Containers

✅ Advantages:

- Use exactly same db engine:
  - transactions, constraints, triggers, ...
- Catch provider-specific bugs early
- Suitable for pipelines

❌ Disadvantages:

- Much slower: container startup time
- More setup complexity (Docker)

ℹ️ Best and recommended approach for EF Core integration testing

- In-Memory
- SQLite
- Test Containers

Testcontainers

Open Source Testing library

- Spins up real Docker containers for your integration tests
- No setup complexity
- Containers start before tests and are removed afterward automatically

https://testcontainers.com/modules/mssql/?language=dotnet

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways:

- Testing Strategies:

Strategy	Best For	Speed	Production Accuracy
In-Memory	Very simple unit tests / demos	Fastest	Not relational
SQLite	Lightweight integration tests	Fast	Close, not exact
Testcontainers	High-confidence integration & CI/CD	Slowest	Exact match

- Always:
  - Create fresh `DbContext` per test (or use shared container with cleanup)
  - Apply migrations: `context.Database.EnsureCreated()` or `Migrate()`
  - Add cleanup/teardown for shared databases

Performance

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Performance

- EF Core simplifies development but can hide performance-critical details (SQL, roundtrips)
- Focus: Achieve good performance and avoid common pitfalls

Identify bottlenecks and measure, measure, measure

- Don't optimize without identifying problems
- Use data to understand where time is spent
- Benchmark your code and alternatives
- Public benchmarks may not reflect your use case

Premature Optimization

Premature optimization is the root of all evil -- DonaldKnuth

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Performance Factors

- **SQL efficiency**: Optimize queries and indexes
- **Data volume**: Load only what's needed, avoid cartesian explosion
- **Roundtrips**: Minimize database calls (latency adds up)
- **EF costs**: Query compilation, change tracking overhead

Monitor EF Core Behavior

- **Log SQL queries**: See what EF generates (`LogTo`, `ToQueryString()`)
- **Track roundtrips**: Watch for N+1 query problems
- **Understand execution timing**: When does EF hit the database?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Cache outside the database

- Minimize database interactions through caching
- Cache results outside the database
- Essential for scalable applications
- Scaling application tier is easier than database tier

BenchmarkDotNet

- Industry-standard library for accurate .NET benchmarking

```
[MemoryDiagnoser]
public class EFCoreBenchmarks {
    private BlogsContext _context;

    [GlobalSetup]
    public void Setup() {
        _context = new BlogsContext();
        _context.Database.EnsureCreated();
    }

    [GlobalCleanup]
    public void Cleanup() => _context.Dispose();

    [Benchmark]
    public List<Blog> LoadAllBlogs_Tracking() => _context.Blogs.ToList();

    // More Benchmark-decorated methods
}
```

- Console App in separate project:

```
// Run benchmarks
public class Program
{
    public static void Main(string[] args)
    {
        BenchmarkRunner.Run<EFCoreBenchmarks>();
    }
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Efficient Querying

Use indexes properly

- Utilize indexes for faster queries
- Check query plans to ensure indexes are used
- Avoid unnecessary indexes to reduce update overhead

```
protected override void OnModelCreating(ModelBuilder modelBuilder) {
    modelBuilder.Entity<Blog>().HasIndex(b => b.Url).IsUnique();
}
```

```
// Matches on start, so uses an index
var posts1 = context.Blogs.Where(p => p.Url.StartsWith("A")).ToList();
// Matches on end, so does not use the index
var posts2 = context.Blogs.Where(p => p.Url.EndsWith("A")).ToList();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Project only what you need

- Use `Select` to query specific columns, not entire entities

```
// ❌ Bad: Fetches all Blog columns
foreach (var blog in context.Blogs)
{
    Console.WriteLine("Blog: " + blog.Url);
}

// ✅ Good: Fetches only Url column
foreach (var blogUrl in context.Blogs.Select(b => b.Url))
{
    Console.WriteLine("Blog: " + blogUrl);
}
```

- Reduces data transfer and memory usage

Limit the resultset size

- Limit query results to avoid loading excessive data
- Implement pagination for large resultsets
- Ensure UI can handle and indicate limited results

```
var blogs25 = context.Posts
    .Where(p => p.Title.StartsWith("A"))
    .Take(25)
    .ToList();
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Buffering and streaming

- Buffering such as `toList()`, loads all results into memory
- Streaming delivers results one at a time
- Use streaming for large resultsets to reduce memory usage

Tracking vs No-Tracking

- Tracking queries maintain a dictionary of tracked instances
- No-tracking queries avoid change tracking overhead
- Use no-tracking for read-only scenarios

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Using SQL queries

- Write optimized SQL for complex queries
- Use SQL queries directly in EF
- Consider user-defined functions and database views

Asynchronous Programming

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Asynchronous Programming

- Async operations free threads during database I/O
- Improves UI responsiveness and web app throughput
- EF Core provides async methods (`SaveChangesAsync`, `ToListAsync`)

Async Method Example

- Example of using `DbContext.SaveChangesAsync`
- Use `async` and `await` keywords

```
public async Task<List<Blog>> GetBlogsWithRecentPostsAsync()
{
    // ToListAsync: Execute query and return list
    var blogs = await context.Blogs
        .Include(b => b.Posts.Where(p => p.PublishedDate > DateTime.Now.AddDays(-7)))
        .Where(b => b.IsActive)
        .ToListAsync();

    return blogs;
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Async LINQ Operators

- EF Core provides async extension methods for LINQ queries:
  - `ToListAsync()`
  - `SingleAsync()`
  - `AsAsyncEnumerable()`
  - `FirstOrDefaultAsync()`
  - …
- No async versions for operators that build LINQ expression trees:
  - `Where()`
  - `OrderBy()`
  - `Select()`
  - `Skip()`
  - …

Efficient Updating

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Batching

- EF Core batches multiple updates in a single roundtrip
- Reduces the number of database roundtrips

```
var blog = context.Blogs.Single(b => b.Url == "https://www.infosupport.com/");

blog.Url = "https://training.infosupport.com/";
context.Add(new Blog { Url = "http://newblog1.microsoft.com" });
context.Add(new Blog { Url = "http://newblog2.microsoft.com" });

context.SaveChanges();    // 👈 One single roundtrip with 3 commands
```

- Batch size can be configured for optimal performance ...

Batching Configuration

- Configure batch size for performance
- Benchmark before modifying batch thresholds

```
protected override void OnConfiguring(DbContextOptionsBuilder optBuilder) {
  optBuilder.UseSqlServer(
    @"Server=(localdb)\mssqllocaldb;Database=Blg;Trusted_Connection=True",
    o => o
      .MinBatchSize(1)
      .MaxBatchSize(100));
}
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Key Takeaways

- Measure First, Optimize Later
  - Use **BenchmarkDotNet** for accurate measurements
  - Monitor SQL queries and roundtrips (`LogTo`, `ToQueryString()`)
- Efficient Querying
  - Use indexes properly - Check query plans, avoid unnecessary indexes
  - Project only what you need - `Select` specific columns
  - Limit resultsets - Use `Take()`
- Tracking & Loading Strategies
  - No-tracking queries
  - Filtered Include - Load subset of related entities
  - Streaming vs Buffering - Stream large resultsets to reduce memory

Key Takeaways

- Async & Batching
  - Async operations - Free threads during I/O (`SaveChangesAsync` ...)
  - Batching - EF Core batches multiple updates in single roundtrip
- Configure batch size - Optimize for your workload (benchmark first)
- Performance Factors
  - SQL efficiency - Optimize queries and indexes
  - Data volume - Load only what's needed
  - Roundtrips - Minimize database calls
  - Caching - Cache outside database for scalability

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Wrap-up

Resources

- Microsoft Learn - Entity Framework Official Documentation
- Github Repo with Samples from the Documentation

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Continued training

- LINQ: .NET Language-Integrated Query
- Multithreading, Parallel Programming and Asynchronous Programming in C# .NET
- Developing SQL Databases
- Querying Data with Transact-SQL
- Advanced Querying Data with Transact-SQL
- Unit Testing In C#
- Docker Essentials

More courses: https://training.infosupport.com/trainingen

Questions?

Last chance!

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

The end

Don't forget to **fill out the evaluation form** at

http://eval

or at

https://tinyurl.com/infosupporteval

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support